

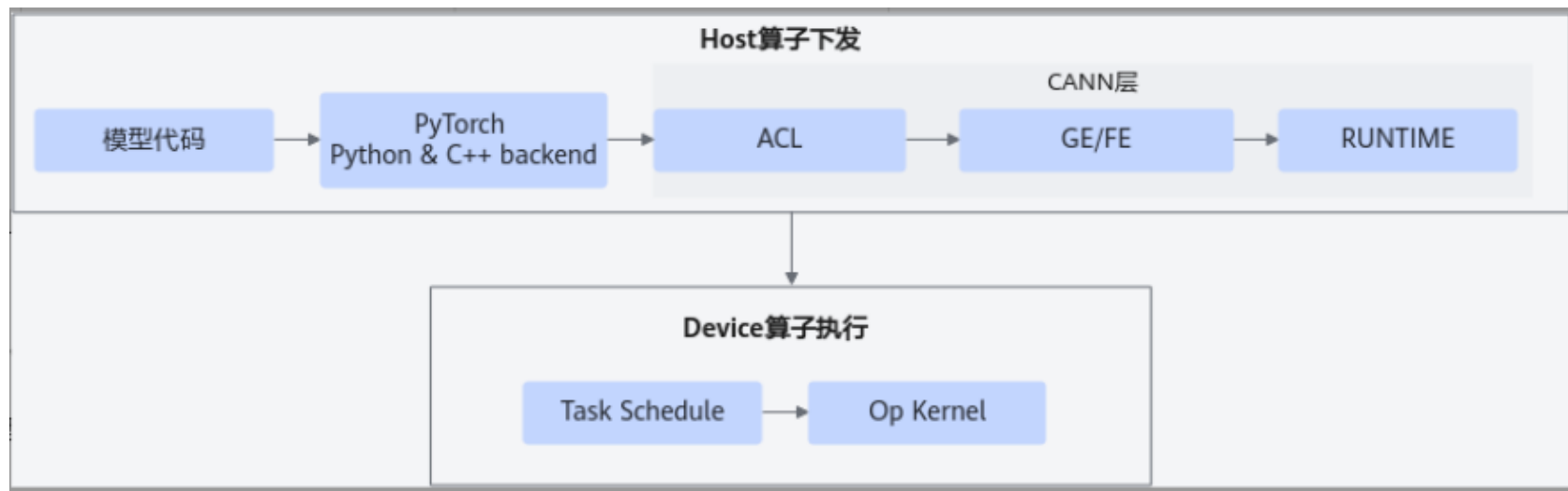
目 录

一、大模型训练性能调优概述

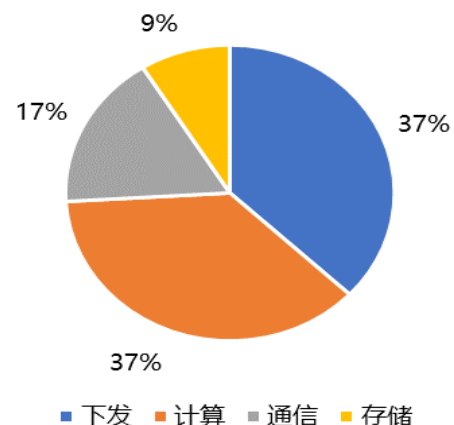
二、Pytorch大模型训练调优工具介绍

三、性能调优问题定位案例

大模型性能优化原则：减少Host算子下发时间和Device算子执行时间



性能瓶颈的发生频率占比



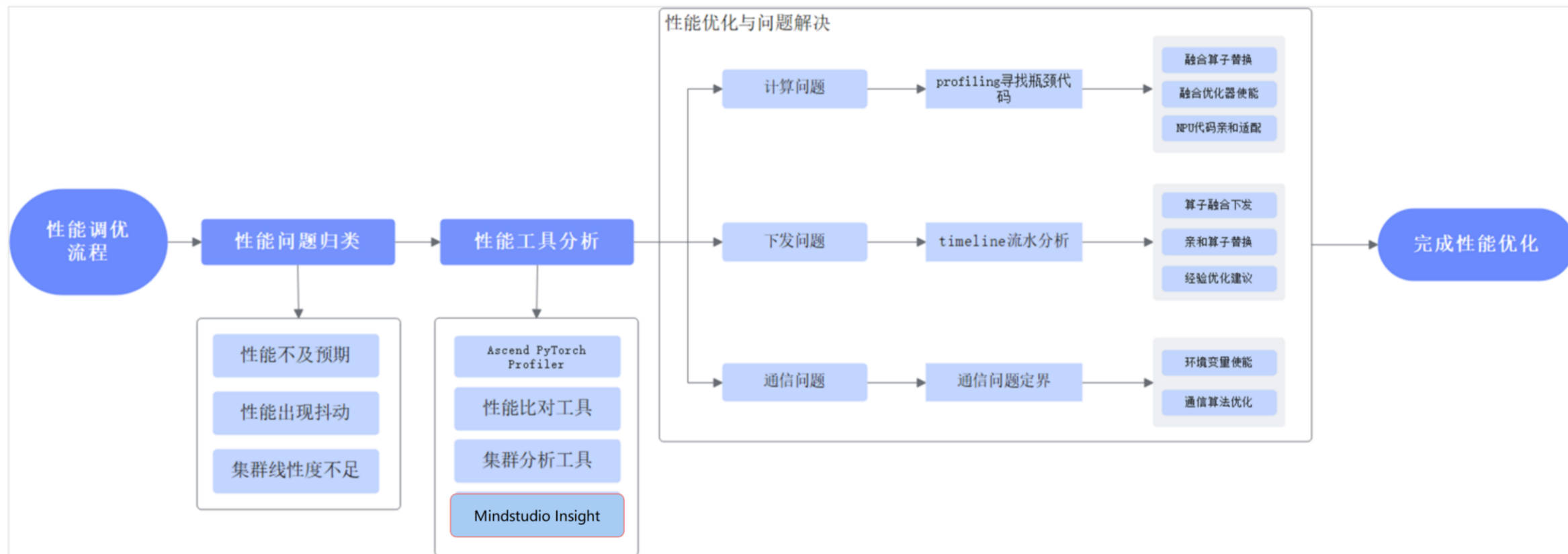
计算与下发是常见性能瓶颈

- CPU只负责算子的下发，而NPU负责算子的执行，算子下发和执行异步发生，性能瓶颈在此过程中体现。
- 性能优化的总体原则为**减少Host算子下发时间**和**减少Device算子执行时间**。

性能调优核心思路：快速排查 + 详细排查

快速排查：确认问题场景，确认性能优化目标（有无标杆对比）；排查近期软硬件变更，优先考虑劣化是否是变更导致。

详细排查：主要围绕下发、通信、计算三类常见问题开展



常见大模型训练性能问题分布

根因分层	根因分类	数量	占比(%)
下发问题	小算子问题	14	16.47
	IO问题	7	8.24
	OS	2	2.35
	内存问题	6	7.06
	环境问题	4	4.71
通信问题	参数配置问题	10	11.76
	网络问题	6	7.06
计算问题	算子泛化问题	18	21.18
	融合算子问题	12	14.12
	硬件问题	6	7.06

性能调优概述小结

SUMMARY



- **性能优化的总体原则：**为减少Host算子下发时间和减少Device算子执行时间
- **性能调优流程：**明确问题 + 工具采集、分析识别 + 性能调优、验证

目 录

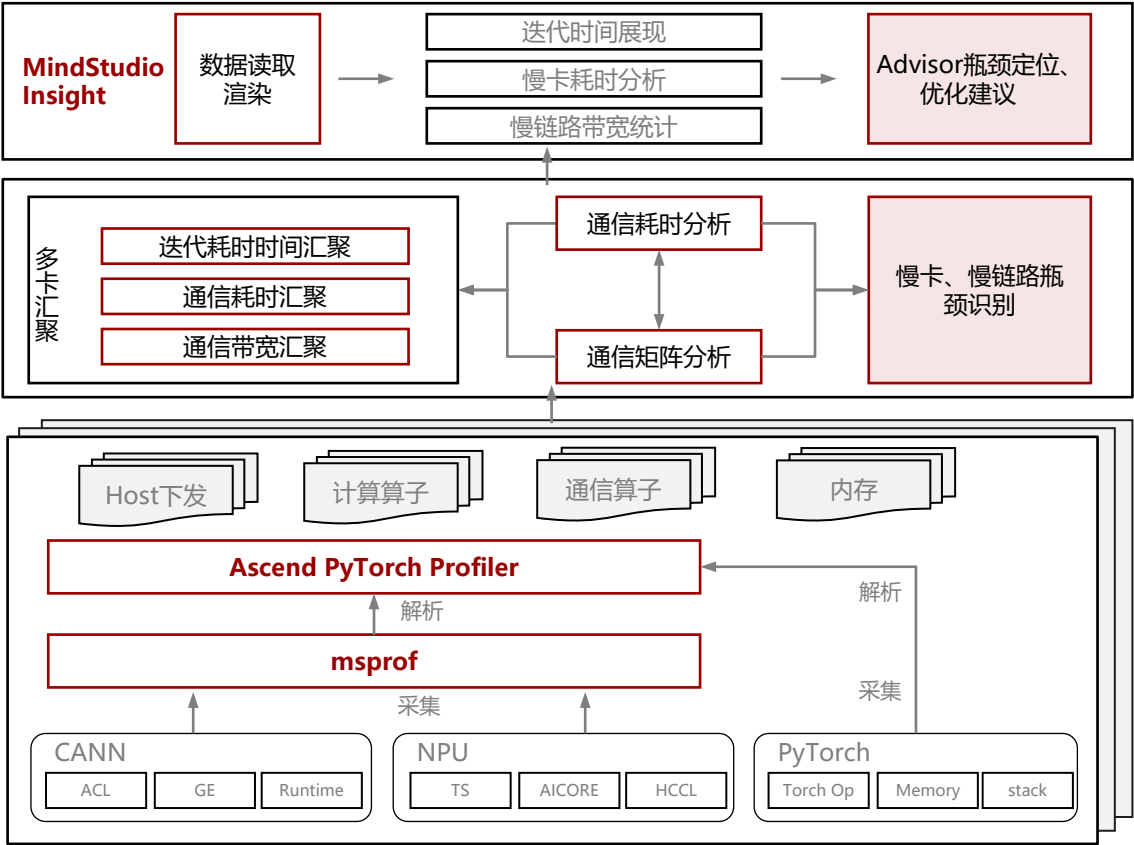
一、大模型训练性能调优概述

二、Pytorch大模型训练调优工具介绍

三、性能调优问题定位案例

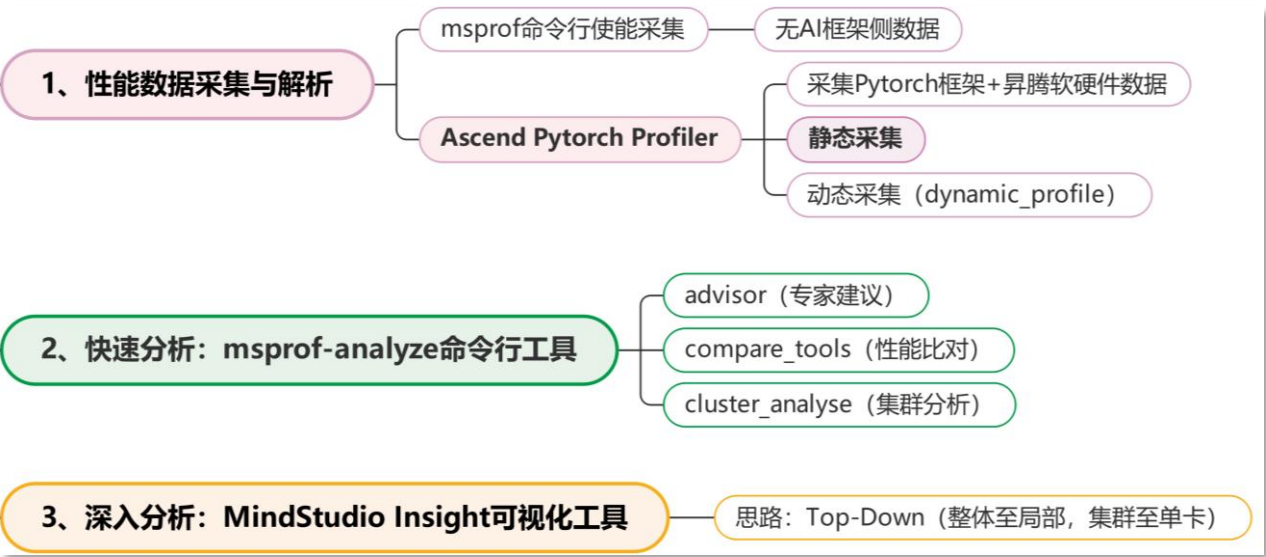
MindStudio 性能调优工具使用流程

MindStudio-性能调优工具架构



采集 → 解析 → 统计 → 瓶颈识别 → 优化推荐

性能调优工具使用流程



目 录

一、大模型训练性能调优概述

二、Pytorch大模型训练调优工具介绍

- ① 数据采集与解析: MS profiler工具
- ② 快速分析: msprof-analyze命令行工具
- ③ 深入分析: MindStudio Insight可视化工具

三、性能调优问题定位案例

Ascend PyTorch Profiler总览：对标框架原生能力，能力完备，推荐使用

对标框架原生能力：

Ascend PyTorch Profiler是针对PyTorch框架开发的性能数据采集和解析工具，通过在PyTorch**训练脚本中插入Ascend PyTorch Profiler接口**，执行训练的同时**采集性能数据**，完成训练后直接**输出可视化的性能数据文件**，提升了性能分析效率。

采集与能力完备：

Ascend PyTorch Profiler接口可**全面采集PyTorch训练场景下的性能数据**，主要包括PyTorch层算子信息、CANN层算子信息、底层NPU算子信息、以及**算子内存占用**信息等，可以全方位分析PyTorch训练时的性能状态。

PyTorch框架算子多级分发 PyTorch框架性能数据

ACL接口执行

GE融合

Runtime

CANN性能数据

Device

NPU性能数据

Ascend PyTorch Profiler采集方式：提供profiling的原生API能力

Ascend PyTorch Profiler完全对标PyTorch-GPU场景下的使用方式，支持采集PyTorch框架和昇腾软硬件数据。该方式提供with语句和start/stop语句两种使能方式，同时用户可以通过设置schedule参数灵活控制不同step的Profiling行为。

采集方式一：通过with语句进行采集

```
import torch_npu
experimental_config = torch_npu.profiler._ExperimentalConfig(
    aic_metrics=torch_npu.profiler.AiCMetrics.PipeUtilization,
    profiler_level=torch_npu.profiler.ProfilerLevel.Level1,
    l2_cache=False
)
with torch_npu.profiler.profile(
    activities=[
        torch_npu.profiler.ProfilerActivity.CPU,
        torch_npu.profiler.ProfilerActivity.NPU
    ],
    record_shapes=True,
    profile_memory=True,
    with_stack=True,
    experimental_config=experimental_config,
    schedule=torch.profiler.schedule(wait=10, warmup=0, active=1, repeat=1),
    on_trace_ready=torch_npu.profiler.tensorboard_trace_handler("./profiling_data")
) as prof:
    # 模型训练代码
    for epoch, data in enumerate(dataloader):
        train_model_one_step(model, data)
        prof.step()
```

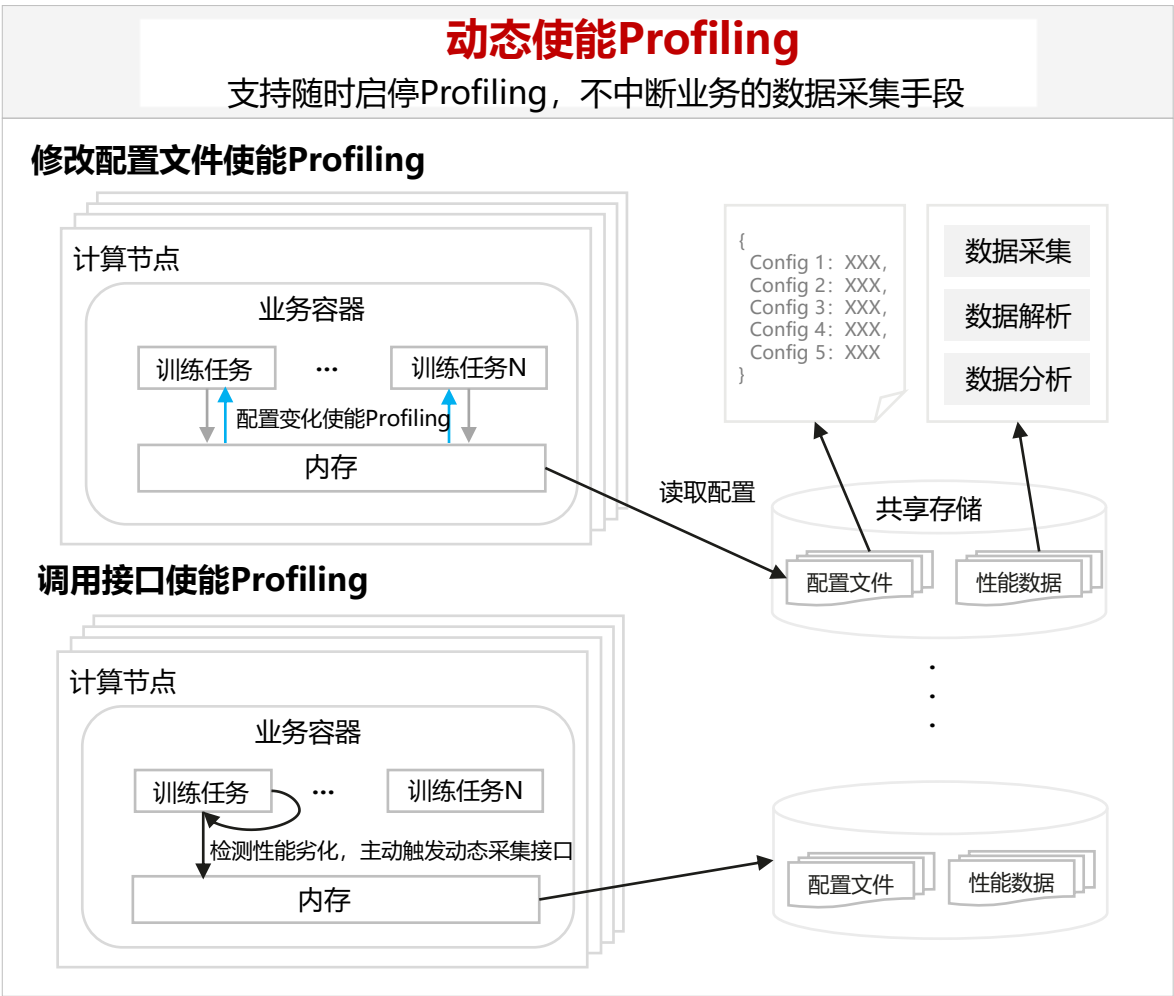
采集方式二：start, stop方式进行采集

```
import torch_npu
experimental_config = torch_npu.profiler._ExperimentalConfig(
    aic_metrics=torch_npu.profiler.AiCMetrics.PipeUtilization,
    profiler_level=torch_npu.profiler.ProfilerLevel.Level1,
    l2_cache=False
)
prof = torch_npu.profiler.profile(
    activities=[
        torch_npu.profiler.ProfilerActivity.CPU,
        torch_npu.profiler.ProfilerActivity.NPU
    ],
    record_shapes=True,
    profile_memory=True,
    with_stack=True,
    experimental_config=experimental_config,
    on_trace_ready=torch_npu.profiler.tensorboard_trace_handler("./profiling_data")
)
# 模型训练代码
for epoch, data in enumerate(dataloader):
    if epoch == 11:
        prof.start()
        train_model_one_step(model, data)
        prof.step()
    if epoch == 12:
        prof.stop()
```

资料链接：https://www.hiascend.com/document/detail/zh/mindstudio/81RC1/T&ITools/Profiling/atlasprofiling_16_0004.html

Ascend PyTorch Profiler采集方式：提供配置文件方式动态使能，不中断业务

何时需要动态采集：长稳训练时，不能一直开着采集，但又不知道该采哪些step；或者想要临时修改采集参数，又不想侵入式修改训练脚本，此时可以用动态采集。



用户需要在自己的脚本中增加以下三条语句：

1. `from torch_npu.profiler import dynamic_profile as dp` 导入动态Profiling的模块
2. `dp.init("[配置文件路径]")` 使用配置文件的存放路径初始化动态Profiling
3. `dp.step()` 手动划分用户脚本的step

配置文件样例

```
1  "activity": ["CPU", "NPU"],
2  "prof_dir": "./prof_result",
3  "analyse": false,
4  "record_shapes": false,
5  "profile_memory": false,
6  "with_stack": false,
7  "with_flops": false,
8  "with_modules": false,
9  "experimental_config": {
10     "profiler_level": "Level0",
11     "aic_metrics": "PipeUtilization",
12     "l2_cache": false,
13     "data_simplification": true,
14     "record_op_args": false,
15     "export_type": "text",
16     "msprof_tx": true
17  },
18  "active": 5
19 }
```

Ascend PyTorch Profiler配置参数：涵盖框架、CANN、Device、内存、堆栈等典型能力

```
import torch_npu
experimental_config = torch_npu.profiler._ExperimentalConfig(
    aic_metrics=torch_npu.profiler.AiCMetrics.PipeUtilization,
    profiler_level=torch_npu.profiler.ProfilerLevel.Level1,
    l2_cache=False
)
with torch_npu.profiler.profile(
    activities=[
        torch_npu.profiler.ProfilerActivity.CPU,
        torch_npu.profiler.ProfilerActivity.NPU
    ],
    record_shapes=True,
    profile_memory=True,
    with_stack=True,
    experimental_config=experimental_config,
    schedule=torch.profiler.schedule(wait=10, warmup=0, active=1, repeat=1),
    on_trace_ready=torch_npu.profiler.tensorboard_trace_handler("./profiling_data")
) as prof:
    # 模型训练代码
    for epoch, data in enumerate(dataloader):
        train_model_one_step(model, data)
        prof.step()
```

activities

CPU、NPU事件采集列表，Enum类型。取值为：

- torch_npu.profiler.ProfilerActivity.CPU：框架侧数据采集的开关。
 - torch_npu.profiler.ProfilerActivity.NPU：CANN软件栈及NPU数据采集的开关。
- 默认情况下两个开关同时开启。

record_shapes

算子的InputShapes和InputTypes，Bool类型。取值为：

- True：开启。
- False：关闭。默认值。

开启torch_npu.profiler.ProfilerActivity.CPU时生效。

profile_memory

算子的内存占用情况，Bool类型。取值为：

- True：开启。
- False：关闭。默认值。

with_stack

算子调用栈，Bool类型。取值为：

- True：开启。
- False：关闭。默认值。

开启torch_npu.profiler.ProfilerActivity.CPU时生效。

on_trace_ready

设置采集结束时执行的操作，Callable类型。通常选择tensorboard_trace_handler，该接口用于解析采集的Profiling数据，

Ascend PyTorch Profiler配置参数：包含PMU采样和数据等级设置

experimental_config

- 该参数用于设置NPU上的一些专有采集配置项。
- profiler_level: 设置NPU采集的Level, 支持:
 - torch_npu.profiler.ProfilerLevel.Level0 (默认)
 - torch_npu.profiler.ProfilerLevel.Level1
 - torch_npu.profiler.ProfilerLevel.Level2
 - data_simplification: 数据精简开关, 默认打开;
 - aic_metrics: AI Core的性能指标采集项 (高阶用户);
 - l2_cache: l2 cache数据采集开关 (性能膨胀);
 - record_op_args: 记录AOE组件算子Dump数据 (不建议与Profiling功能一起使用)

AicMetrics	Metrics数据
PipeUtilization	计算单元和搬运单元耗时占比, 为默认值。
ArithmeticUtilization	各种计算类指标占比统计
Memory	外部内存读写类指令占比
MemoryL0	L0内存读写类指令占比
ResourceConflictRatio	流水线队列类指令占比
MemoryUB	内部内存
L2Cache	读写cache命中次数和缺失后重新分配次数

ProfilerLevel	Profiling数据
Level0	框架侧及Device侧算子执行耗时, 为默认值。
Level1	Level0基础上增加CANN软件栈中AscendCL接口、HCCL通信及AI Core的性能指标数据
Level2	Level1基础上增加CANN软件栈的Runtime数据及AI CPU数据

1. 如何选择性能采集工具：

常规场景使用Ascend PyTorch Profiler根据需要采集指定性能项和step，对于启停成本大场景可采用动态启停、在线监控

2. 典型场景参数配置：

数据采集会增加额外开销，特别是with_stack、Level2、L2 cache、record_shapes等开关，非必要不打开。

1. 常规性能分析场景： profiler_level=l1, aic_metrics保持默认PipeUtilization，activities设置采集CPU和NPU。其他开关根据需要开启。

2. 定位代码位置场景： 如需要定位异常算子代码位置，可在常规场景下开启with_stack或with_modules开关开启调用栈（非必要不开启，性能会膨胀）

3. 分析多卡间通信场景： 配置profiler_level=l1及以上。

4. NPU/GPU对比场景： profiler_level=l0, activities设置仅采集NPU或CPU和NPU（根据需要），其他开关关闭

目 录

一、大模型训练性能调优概述

二、Pytorch大模型训练调优工具介绍

- ① 数据采集与解析：MS profiler工具
- ② **快速分析：msprof-analyze命令行工具**
- ③ 深入分析：MindStudio Insight可视化工具

三、性能调优问题定位案例

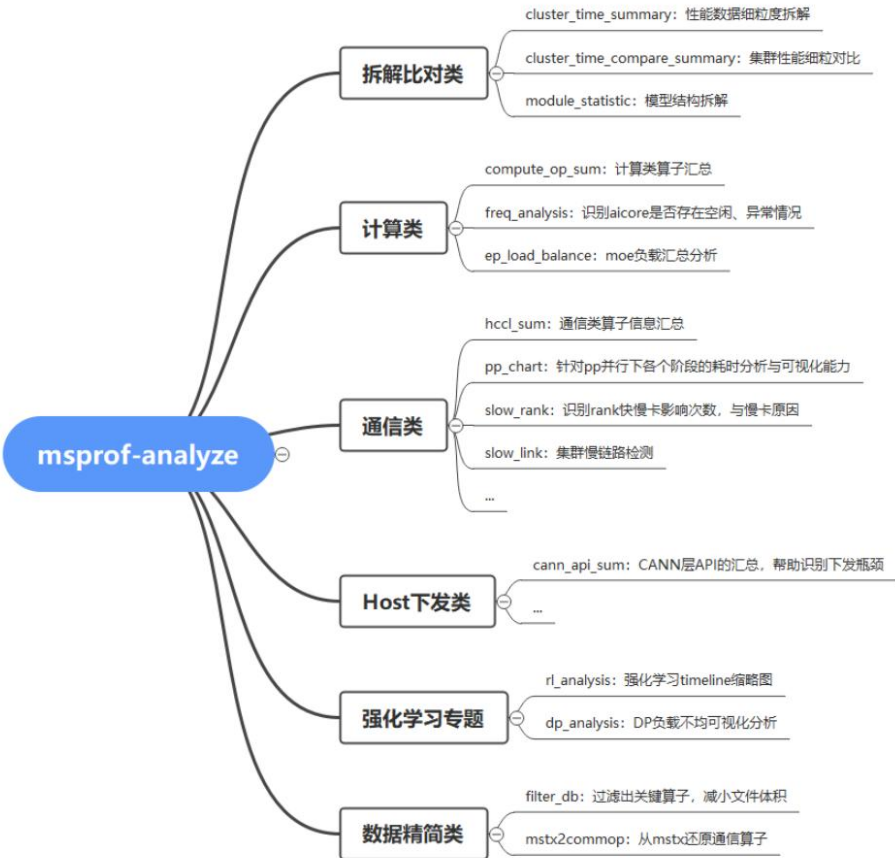
msprof-analyze性能分析：基于性能数据的分析底座和能力集

代码仓：https://gitcode.com/Ascend/mstt/tree/master/profiler/msprof_analyze/（待建立独立新仓）

安装方法：pip install msprof-analyze(或者pip3 install ./msprof_analyze-{version}-py3-none-any.whl)

功能定位：基于采集profiling数据的性能分析底座。

分析能力：



□ 使用方法：

msprof-analyze -m [feature_option] -d <profiling_path> [global_option] [analyze_option]

例如：-m分析能力选项传入cluster_time_summary分析能力，-d参数传入cluster_data性能数据文件夹，结果保存在cluster_data/cluster_analysis_output
msprof-analyze -m cluster_time_summary -d ./cluster_data

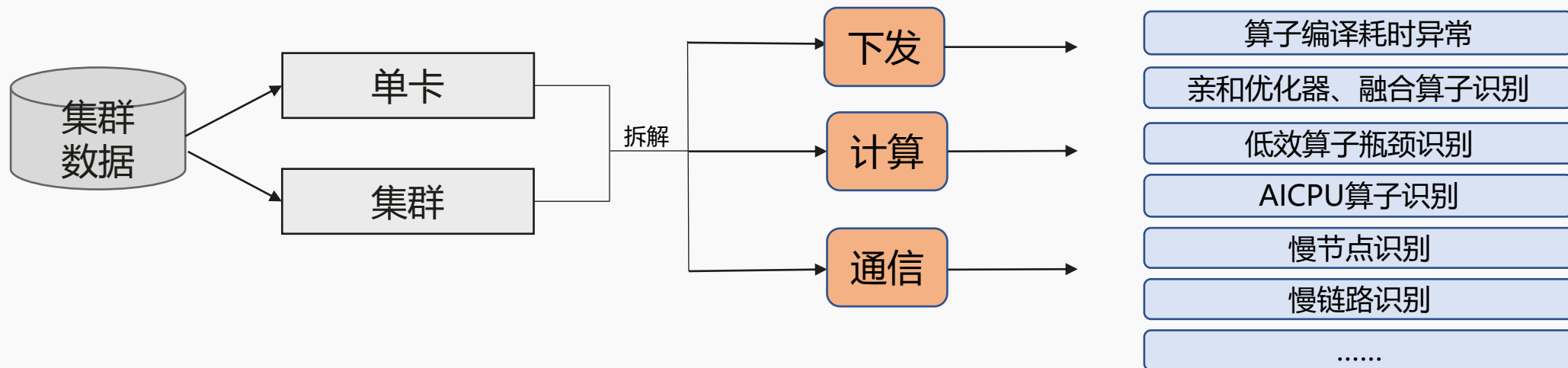
□ 快慢卡分析能力

功能	能力
cluster_time_summary	集群性能数据细粒度拆解
hccl_sum	通讯算子汇总
slow_rank	通过投票机制找出影响最大的慢卡
compute_op_sum	计算类算子汇总
cann_api_sum	CANN层API汇总
...(recipe模式，支持用户自定义开发)	

msprof-analyze advisor功能：自动定位性能瓶颈及优化建议



mstt_advisor_20250625171014.html



Msprof-analyze的advisor功能可以端到端帮用户分析profiling数据，识别性能瓶颈点并能给出优化建议，覆盖集群、单卡场景的下发、计算、通信等20+条建议，使用方法十分简单，如下：

- 1 采集profiling数据
- 2 下载安装msprof_analyze*.whl包
- 3 一键生成分析报告

msprof-analyze advisor all -d [待分析性能数据文件所在路径] -bp [基准性能数据文件所在路径]

Optimization Priority: High Medium Low

overall

comparison

performance problem analysis

memory

Memory Operator Issues

Analysis of rank 0. 发现了572个AscendCL@aclrtFreePhysical算子，花费658817.36399999994us，这将导致大量的空闲时间。

Suggestions

1. For AscendCL@aclrtFreePhysical: 在训练时执行'npu-smi info'观察HBM使用情况，如果达到HBM使用的最大值，请减小您的批大小/微批大小。
2. For AscendCL@aclrtFreePhysical: 首先使用参数'with_stack=True'采集Profiling，然后在trace_view.json中搜索'empty_cache'或'emptyCache'。如果存在，根据trace_view.json中相关事件的调用堆栈删除'torch.cuda.empty_cache()'或'torch.npu.empty_cache()'等代码。

发现大量内存重整操作，建议用户在训练时执行'npu-smi info'观察HBM使用情况

compare_tools(性能拆解比对)

目标:

将训练耗时拆分为算子执行、通信(非计算掩盖部分)、调度、内存4个维度，直接确认主要性能瓶颈。

使用方法:

msprof_analyze compare -d {path1} -bp {path2}即可

Model Profiling Time Distribution									
	Cube Time(Num)	Vector Time(Num)	Flash Attention Time(Forward)(Num)	Flash Attention Time(Backward)(Num)	Computing Time	Mem Usage	Uncovered Communication Time	SDMA Time(Num)	Free Time
GPU	0.839s (899)	0.358s (5767)	0.002s (64)	0.002s (96)	1.202s	31.17G	0.059s	0.010s (1173)	0.099s
NPU	0.707s (899)	0.302s (3786)	0.064s (64)	0.160s (32)	1.234s	31.80G	0.536s	0.012s (229)	1.125s

展示效果:

Base Profiling: C:\Users\s00809967\AppData\Roaming\Space		Comparison Profiling: C:\Users\s00809967\AppData\Roaming\Space\Desktop\UserData\s00809967\ReceiveFile\con						
Index	Duration(ms)	Duration Ratio	Number	Duration(ms)	Duration Ratio	Number	Diff Duration(ms)	Diff Ratio
Computing Time	2,513.15	38.34%	25,256	2,658.53	94.84%	25,652	145.38	105.78%
Conv	1,217.66	18.57%	5,790	976.74	34.84%	3,461	-240.92	80.21%
Conv (Forward) (Cube)	279.83	4.27%	574	344.22	12.28%	732	64.39	123.01%
Conv (Forward) (Vector)	202.36	3.09%	1,754	126.96	4.53%	1,038	-75.40	62.74%
Conv (Backward) (Cube)	441.88	6.74%	784	389.77	13.90%	626	-52.11	88.21%
Conv (Backward) (Vector)	293.59	4.48%	2,678	115.79	4.13%	1,065	-177.80	39.44%
Matmul	14.81	0.23%	2,222	45.12	1.61%	1,588	30.31	304.72%
Matmul (Cube)	7.26	0.11%	1,008	41.11	1.47%	1,016	33.84	566.00%
Matmul (Vector)	7.55	0.12%	1,214	4.02	0.14%	572	-3.53	53.22%
Vector	1,250.96	19.08%	16,984	1,606.18	57.30%	20,425	355.22	128.40%
Vector (Trans)	188.72	2.88%	1,286	452.12	16.13%	3,422	263.40	239.57%
Vector (No Trans)	1,062.24	16.20%	15,698	1,154.06	41.17%	17,003	91.82	108.64%
Cube	15.54	0.24%	64	15.41	0.55%	64	-0.13	99.16%
SDMA (Tensor Move)	12.36	0.19%	196	11.09	0.40%	114	-1.27	89.75%
Other	1.83	0.03%	/	3.99	0.14%	/	2.17	218.78%
Uncovered Communication Time	0.00	0.00%	/	0.43	0.02%	/	0.43	INF
Transmit	0.00	0.00%	/	0.43	0.02%	/	0.43	INF
Free Time	4,042.60	61.66%	/	144.20	5.14%	/	-3,898.40	3.57%
SDMA	0.00	0.00%	/	5.50	0.20%	/	5.50	INF
Free	4,042.60	61.66%	/	138.70	4.95%	/	-3,903.90	3.43%
E2E Time	6,555.75	100.00%	/	2,803.16	100.00%	/	-3,752.58	42.76%

msprof-analyze细粒度拆解比对功能：典型案例

客户现场某次升级CANN包和PTA包后，发生OOM情况，于是采集升级前后两次的profiling，根据算子级比对发现是aten::group_norm多申请了10GB+内存。

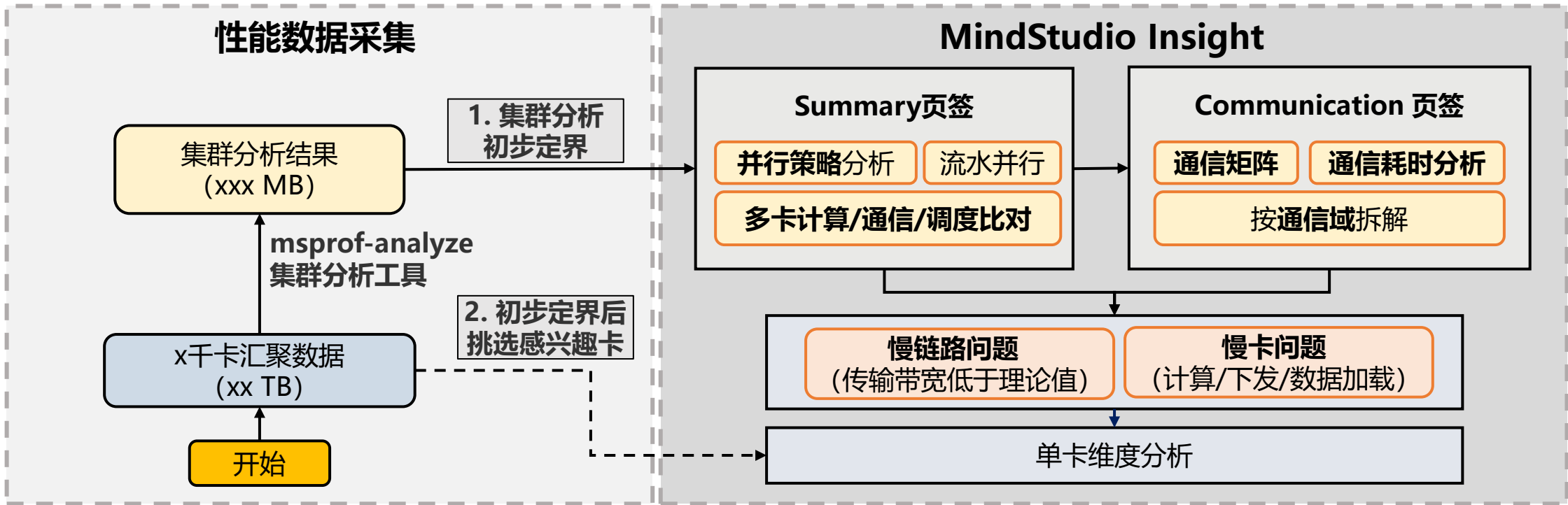
Base Profiling: /data/zyr/diff0828/prof_res/prof_0829_fp16_bz24_cann0825_baohe_debug_active2/localhost.localdomain_2350908_20230829093502_a				Comparison Profiling: /data/zyr/diff0828/prof_res/prof_0829_fp16_bz24_fa_cann013_baohe_debug/localhost.localdomain_2188335_20230829092348				
Operator Name	Input Shape / OP Name	Input Type / Release Time	Size(KB)	Operator Name	Input Shape / OP Name	Input Type / Release Time	Size(KB)	DIFF: (sum(comparison)-sum(base))
aten::empty				aten::empty				
aten::empty				aten::empty				
aten::random_				aten::random_				
aten::item				aten::item				
enumerate(DataLoader)#_SingleProcessDataLoaderIter._next_				enumerate(DataLoader)#_SingleProcessDataLoaderIter._next_				
aten::to			165888.5	aten::to			165888.5	0.00%
aten::to	aten::empty_strided	165888.5	66205313.89	aten::to	aten::empty_strided	165888.5	63123608.92	0.00%
aten::to	aten::empty_strided	14.5	66184591.43	aten::to	aten::empty_strided	14.5	63097932.93	
enumerate(DataLoader)#_SingleProcessDataLoaderIter._next_				enumerate(DataLoader)#_SingleProcessDataLoaderIter._next_				
aten::to			83968	aten::to			83968	0.00%
aten::conv2d	acInnCast	83968	436624.48	aten::conv2d	acInnCast	83968	466612.49	
aten::conv2d			3538944.5	aten::conv2d			3538944.5	0.00%
aten::group_norm	Conv2D	3538944.5	162485.88	aten::group_norm	Conv2D	3538944.5	50678.81881	-44.44%
aten::group_norm			31850537	aten::group_norm			17694757.5	
aten::native_batch_norm		3.5	3232.230011	aten::native_batch_norm		3.5	3318.089996	
aten::native_batch_norm		3.5	3256.530070	aten::native_batch_norm		3.5	3237.770000	
aten::native_batch_norm		3538944.5	4432.809998	aten::native_batch_norm		3538944.5	4498.860016	
aten::native_batch_norm		3.5	28357.84	aten::native_batch_norm		3.5	15895.35001	
aten::native_batch_norm		3.5	28325.42999	aten::native_batch_norm		3.5	15888.85001	
acInnAddcmul		14155779.5	30.44000244	acInnAddcmul		3538944.5	8262.920013	
acInnAddcmul		3538944.5	20799.90997	acInnInplaceOne		3.5	9147.810028	
acInnInplaceOne		3.5	9115.839996	acInnInplaceOne		3.5	7691.440002	
acInnInplaceOne		3.5	7580.959991	acInnInplaceOne		3.5	7633.440002	
acInnInplaceOne		3.5	7519.5	Identity		3538944.5	197.960022	
Identity		3538944.5	138.0200195	Identity		3538944.5	88.19000244	
Identity		3538944.5	94.45999146	aten::empty		3538944.5	9198.580017	
aten::empty		3538944.5	21723.72	acInnInplaceZero		3.5	10736.37	
acInnInplaceZero		3.5	10761.82999	acInnInplaceZero		3.5	7761.850006	
acInnInplaceZero		3.5	7658.220001	acInnInplaceZero		3.5	7590.309998	
acInnInplaceZero		3.5	7436.320007					
aten::silu			3538944.5	aten::silu			3538944.5	0.00%
aten::conv2d	Swish	3538944.5	335.3200073	aten::conv2d	Swish	3538944.5	264.4200134	
aten::conv2d			3538944.5	aten::conv2d			3538944.5	0.00%
aten::group_norm	Conv2D	3538944.5	40320.29999	aten::group_norm	Conv2D	3538944.5	4035.299979	
aten::group_norm			31850537	aten::group_norm			17694757.5	-44.44%
aten::native_batch_norm		3.5	302.013808	aten::native_batch_norm		3.5	3160.773939	
aten::native_batch_norm		3.5	319.6499939	aten::native_batch_norm		3.5	3192.860016	
aten::native_batch_norm		3538944.5	442.75	aten::native_batch_norm		3538944.5	3298.460022	
aten::native_batch_norm		3.5	40027.55002	aten::native_batch_norm		3.5	3668.279999	
aten::native_batch_norm		3.5	40026.24002	aten::native_batch_norm		3.5	3665.970001	
acInnAddcmul		14155779.5	45.54998779	acInnAddcmul		3538944.5	215.1500244	
acInnAddcmul		3538944.5	39536.25	acInnInplaceOne		3.5	3580.279999	
acInnInplaceOne		3.5	803.9100037	acInnInplaceOne		3.5	3419.490021	
acInnInplaceOne		3.5	597.5	acInnInplaceOne		3.5	3375.579987	

cluster_analyse(集群分析)

使用场景:

集群场景下，可通过此工具来提取集群迭代耗时和通信数据，快速定位慢卡、慢节点以及慢链路问题，可结合MindStudio Insight可视化工具使用。

- 卡数**较少**时，可直接导入全量prof数据，由可视化工具调用cluster_analyse；
- 卡数**较多**、全量Prof过于笨重时，推荐以命令行方式手动调用cluster_analyse，用insight打开其交付件cluster_analysis_output。



老师敲黑板

msprof-analyze工具常用场景：

Advisor分析：自动化分析优化工具，常见性能问题自动识别，减少调优工作量

Compare分析：开箱性能分析中对比GPU/NPU性能；模型优化前后/正常卡或异常卡性能对比

Cluster分析：集群分析必备，确认后续分析方向，可结合MindStudio Insight可视化工具使用。

更多功能（计算分析/moe负载汇总/Host下发诊断/强化学习专题/ ... ），欢迎探索！

目 录

一、大模型训练性能调优概述

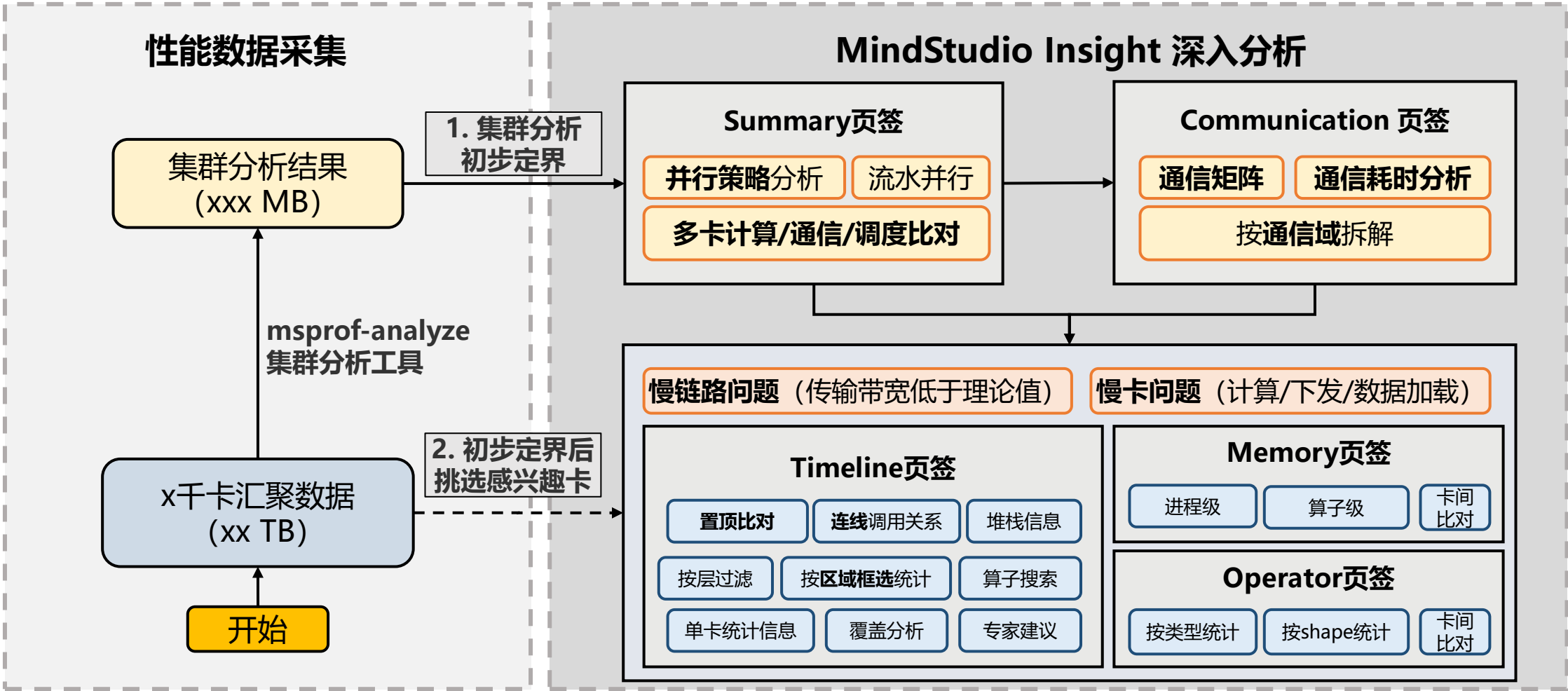
二、Pytorch大模型训练调优工具介绍

- ① 数据采集与解析：MS profiler工具
- ② 快速分析：msprof-analyze命令行工具
- ③ **深入分析：MindStudio Insight可视化工具**

三、性能调优问题定位案例

深入分析：利用MindStudio Insight可视化全量Profiling数据

使用场景： MindStudio Insight能将全量Profiling数据可视化呈现，帮助用户进一步理解与确认问题根因。利用该工具分析问题的流程化思路如下：



Summary页签

快速定位慢卡

并行策略分析：

卡数较少时：选择全展开维度

卡数较多时：选择折叠维度

计算/通信概览：

多卡计算/通信/调度比对

并行策略分析

算法②: Megatron-LM (tp-cp-ep-dp-...

PP大小: 1

TP大小: 16

CP大小: 1

DP大小: 1

EP大小: 1

生成

并行维度②: DP

DP + PP

DP + PP + TP

性能指标: TP-通信时间

筛选范围 (μs): 4112291.044 ~ 4550008.438

目标编号:

查找

3 可选择感兴趣的性能指标。快慢卡分析时，一般关注特性并行域下的通信时间。

张量并行

4112291.044

4550008.438

0

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

5 并行策略正确配置时，可得到慢卡专家建议分析

专家建议: 请关注如下分组是否拖累了整网。同一通信域内，同步耗时越久，说明集群快慢卡不同步问题越严重。

应关注Top3分组/慢卡	TP-卡间同步耗时 (us)
tp2 序号(2)	437717.394
tp14 序号(14)	435075.358
tp13 序号(13)	433863.896

Total 3 items < 1 > 10 条/页

典型case1：计算负载不均衡

计算/通信概览

迭代ID: All 通信域: All 排序方式: 卡序号 前: All

专家建议: Computing has some issues, because the max difference of "Computing" has reached 49775.600000us.
Communication has some issues, because the max difference of "Communication(Not Overlapped)" has reached 96960.960000us.
Free has some issues, because the max difference of "Free" has reached 441197.340000us.

典型case2：下发瓶颈

计算/通信概览

迭代ID: All 通信域: All 排序方式: 卡序号 前: All

专家建议: Communication has some issues, because the max difference of "Communication(Not Overlapped)" has reached 3844542.960000us.
Free has some issues, because the max difference of "Free" has reached 3823694.320000us.

MindStudio Insight可视化

Summary页签

Communication页签

Timeline页签

Memory页签

Operator页签

Summary页签 ——快速定位慢卡

TIP1：左键通信域连线/并行分组框线，按通信域分析



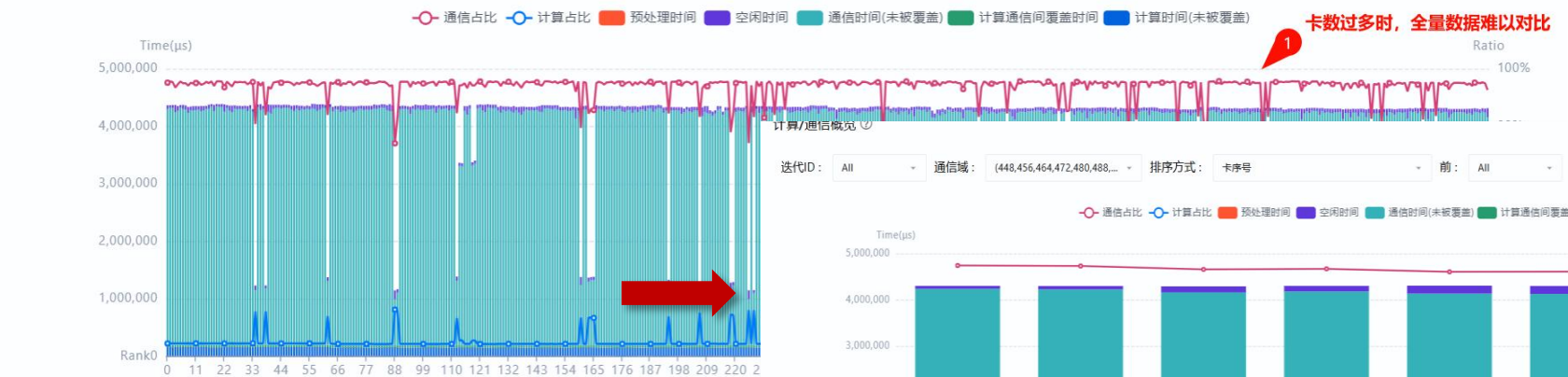
专家建议: 请关注如下分组是否拖累了整网。同一通信域内，同步耗时越久，说明集群快慢卡不同步问题越严重。

应关注Top3分组/慢卡	DP-组间同步平均耗时 (us)	TP-卡间同步耗时 (us)
dp3-pp1-tp0 序号(88)	3202447.487	3307661.86
dp4-pp3-tp1 序号(225)	3198402.21	3299071.06
dp0-pp6-tp7 序号(391)	3217499.166	3272371.029

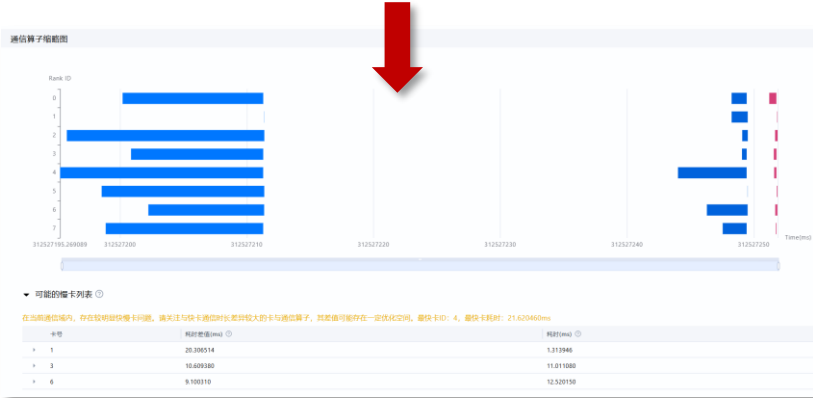
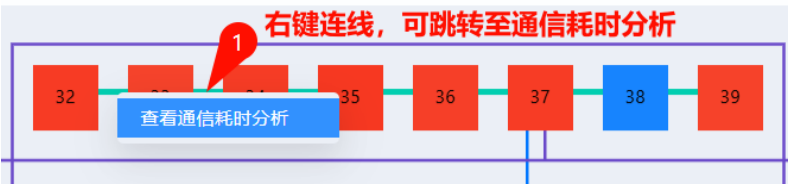
Total 3 items < 1 > 10 条/页

计算/通信概览

迭代ID: All 通信域: All 排序方式: 卡序号 前: All



TIP2：右键连线，可跳转至通信耗时分析



Communication页签 – 异常通信算子分析

Communication页签

——通信矩阵 & 通信耗时分析

时间线内存算子概览通信

迭代ID: 8050通信域: dp:(0, 1, 2, 3, 4, 5, 6, 7, 8, ...算子名称: Total Op Info

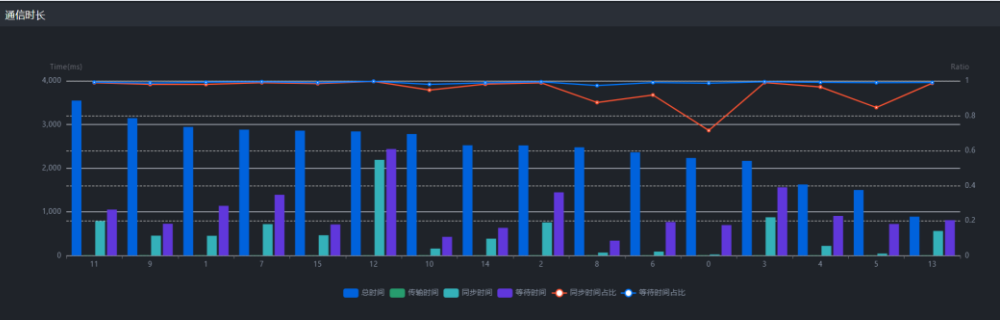
通信矩阵通信耗时分析

此处切换矩阵视图 or 耗时分析视图

若Summary中显示通信时间过长，确认是慢卡还是慢链路。

- ① 首先确认传输时间在通信时间中的占比是否过高。
- ② 占比过高，再通过通信矩阵分析。

传输时间过长 → 慢链路 vs 等待/同步时间过长 → 慢卡



常见慢链路原因：交换机通信拥塞导致通信重传、通信小包、SDMA字节未对齐等（可结合advisor报告分析）。

传输时间占比过高，通过通信矩阵热力图进一步确认是否存在带宽异常



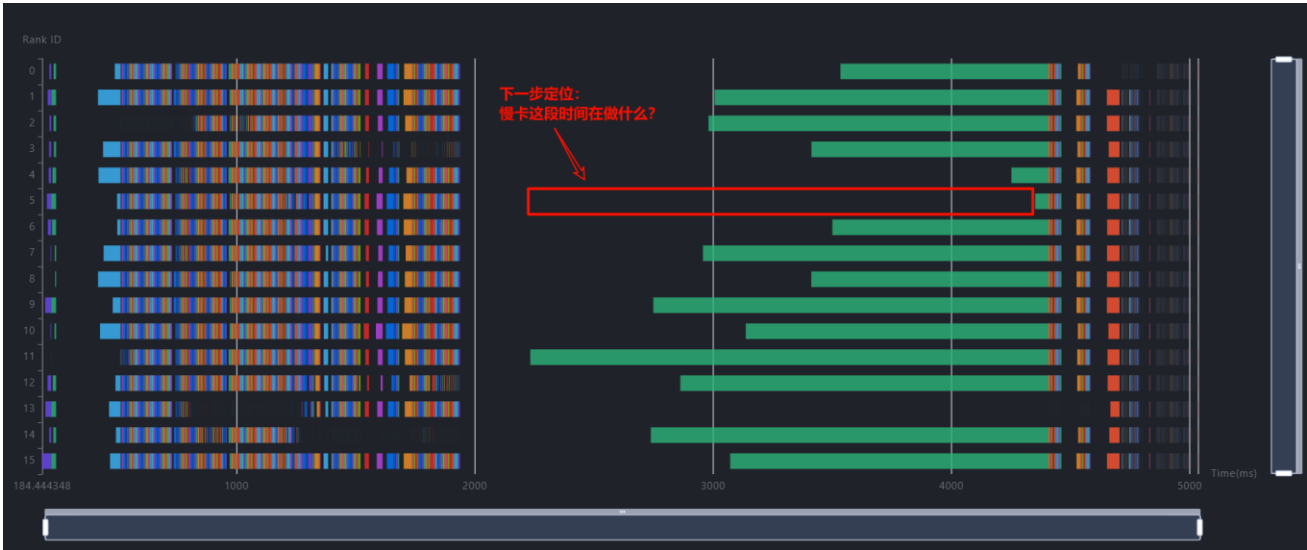
Communication页签

——通信算子平铺缩略图：快速定位快慢卡差异来源

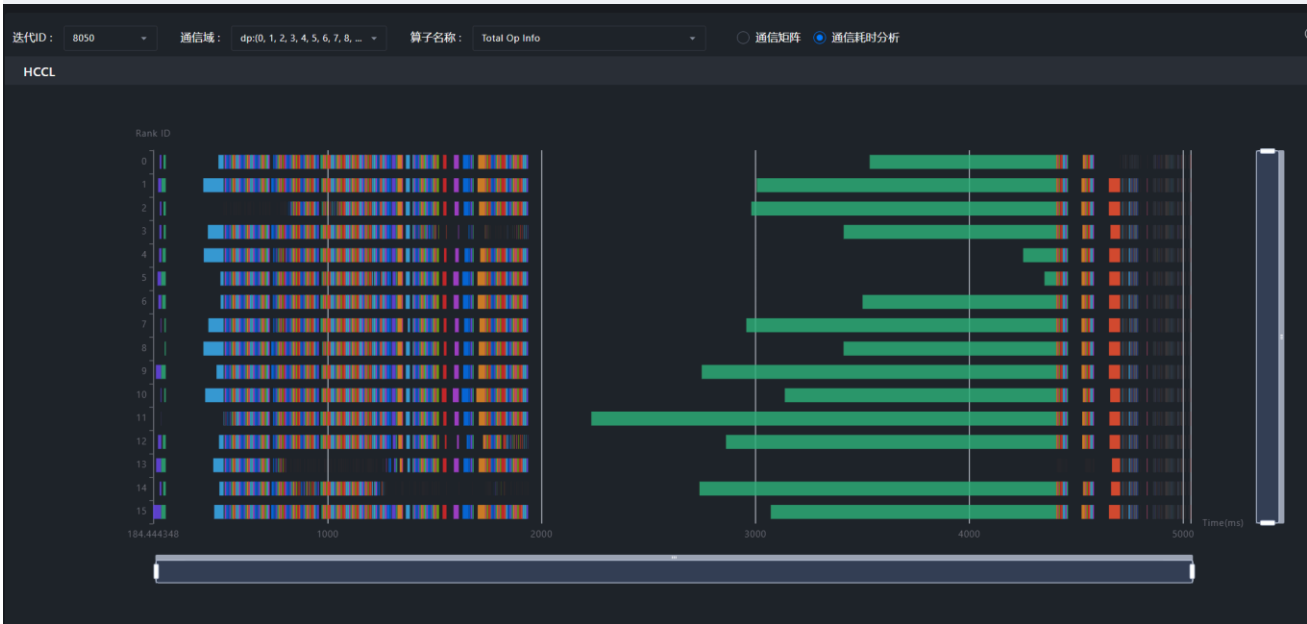
若传输时间占比低，等待/同步时间占比高，则为“快慢卡问题”

通信算子按时间横向平铺，快速定位快慢卡差异来源。

下一步定位目标：分析慢卡在空白时间做什么？需至“时间线”模块，查看具体差异点。



TIPS: “通信” - “时间线” 两页签根据通信算子互相跳转，确认差异来源

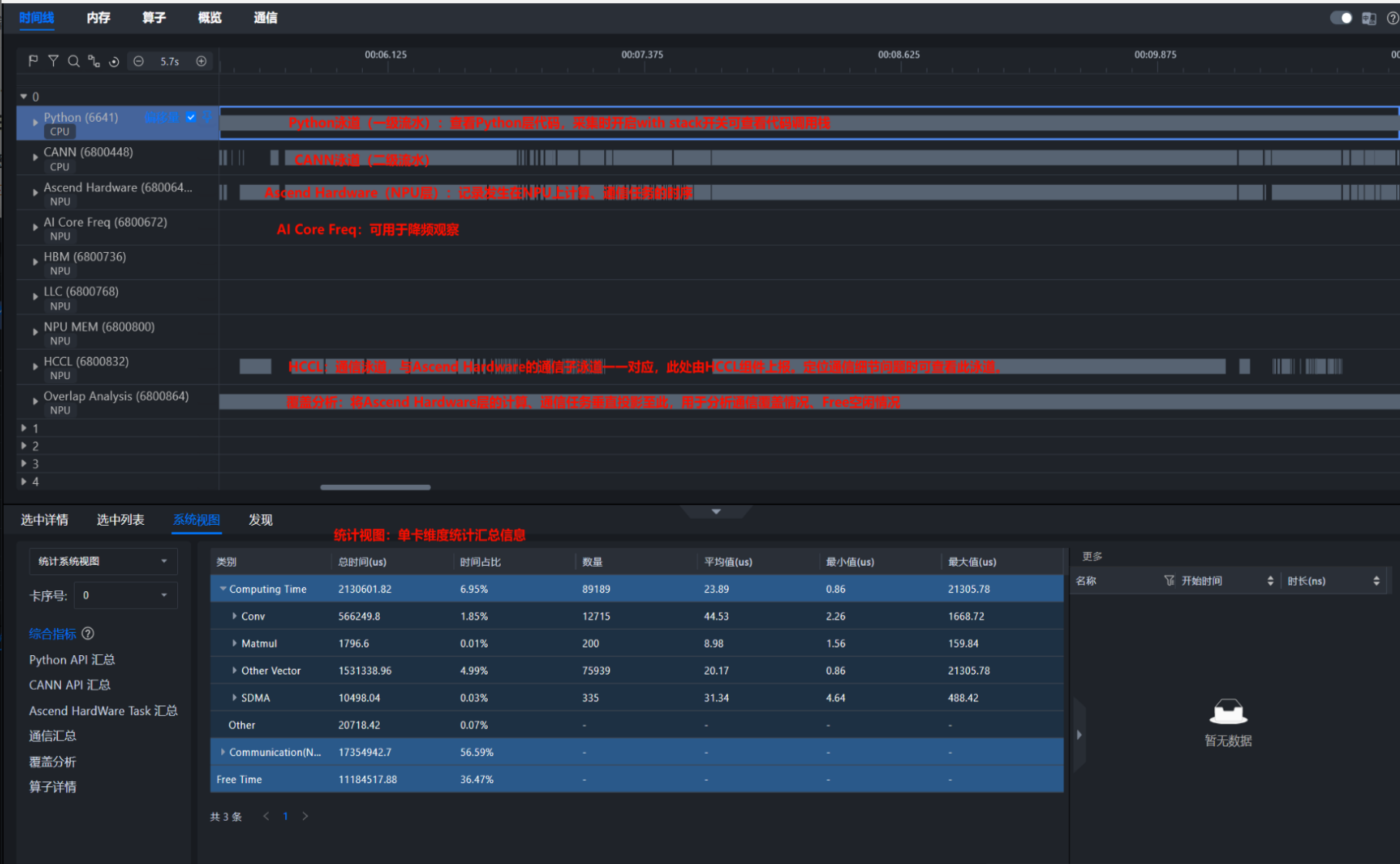


TIPS: Summary页面，右键连线，直接跳转至感兴趣的通信域



Timeline页签 ——Host/Device协同分析，进一步定位根因

- 常用功能
- 置顶对比
 - 覆盖分析
 - Host\device下发
连线关系
 - 按层过滤
 - 堆栈信息
 - 区域框选统计
 - 算子搜索
 - 单卡统计信息



Timeline页签 ——Host/Device协同分析，进一步定位根因

常用功能

➤ 置顶对比

➤ 覆盖分析

➤ Host\device下发
连线关系

➤ 按层过滤

➤ 堆栈信息

➤ 区域框选统计

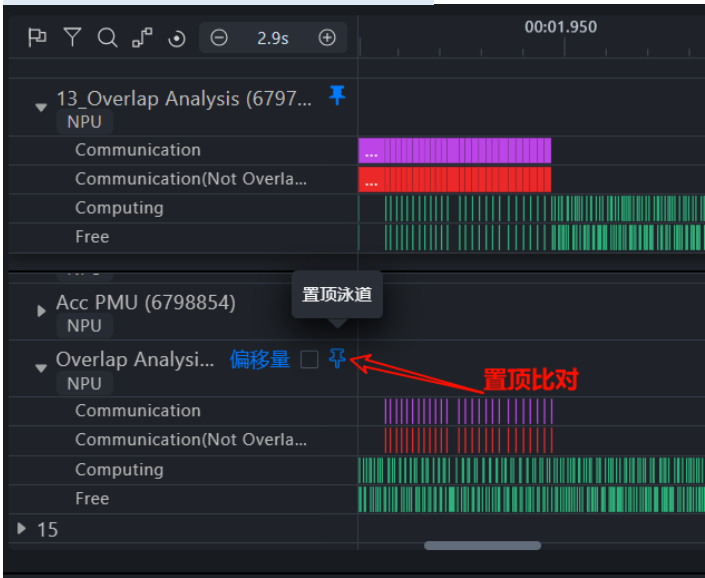
➤ 算子搜索

➤ 单卡统计信息

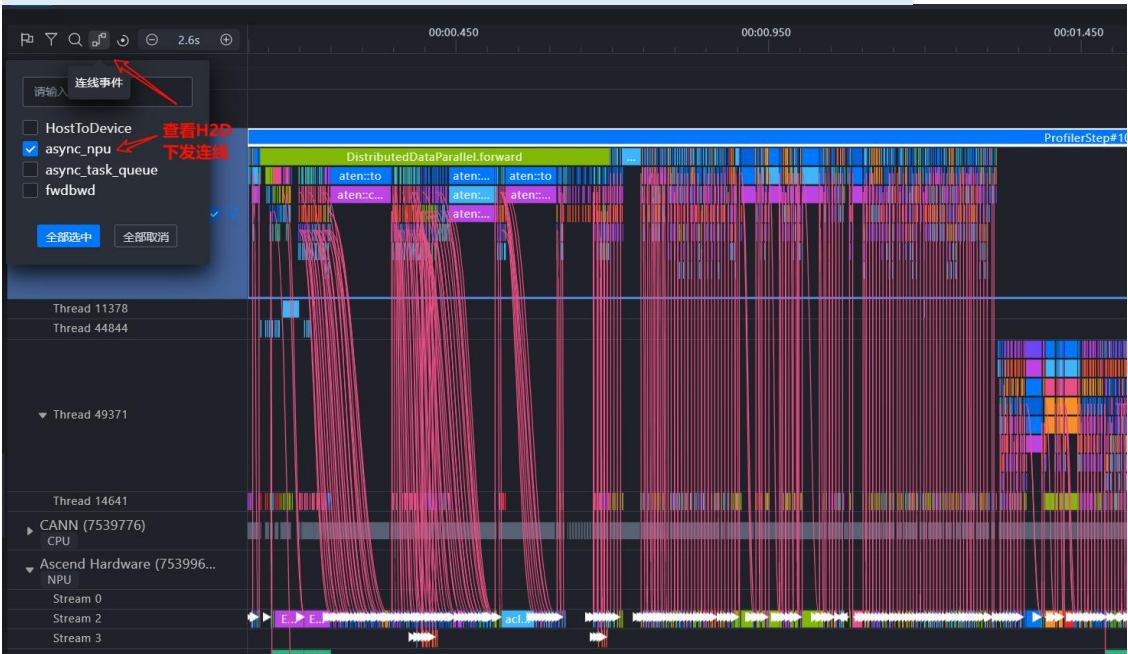
快慢卡定位思路

1. 通过覆盖分析、置顶比对，确认Ascend Hardware层（NPU层）差异来源
2. 通过连线关系，由NPU层向上寻找，确认Host层差异来源

置顶比对 + 覆盖分析



Host2Device下发连线关系 (用于确认调用关系、观察下发瓶颈等)



Memory页签 —— 观察NPU内存利用率

Memory页签

进程级

算子级

卡间对比

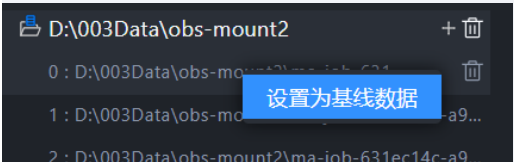
优化思路：尽量增大Batchsize，最大化利用NPU内存

🔍 观察内存趋势，消除尖刺，削峰填谷

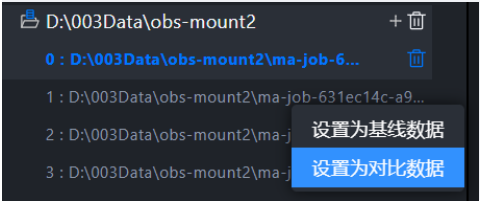
支持开启卡间对比

Step1：两卡Profiling移至同一父目录，Insight打开该父目录

Step2：右键子目录-选择基线数据



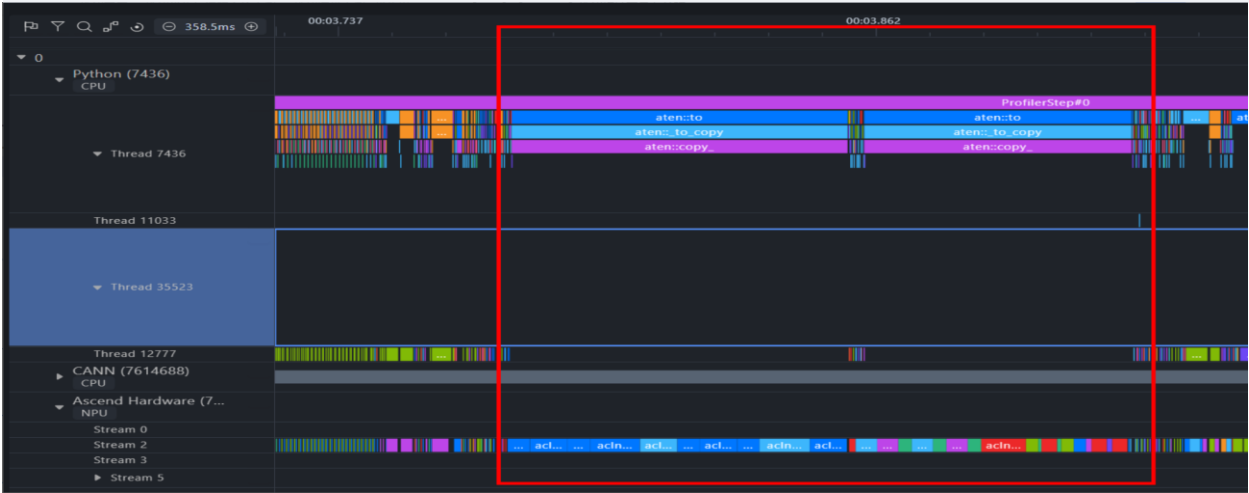
Step3：选择对比数据



Case：NPU利用率不足，观察到存在内存尖刺



跳转至时间线，定位至具体代码。和模型开发人员沟通确认有无优化空间。



Operator页签

——观察Top耗时算子

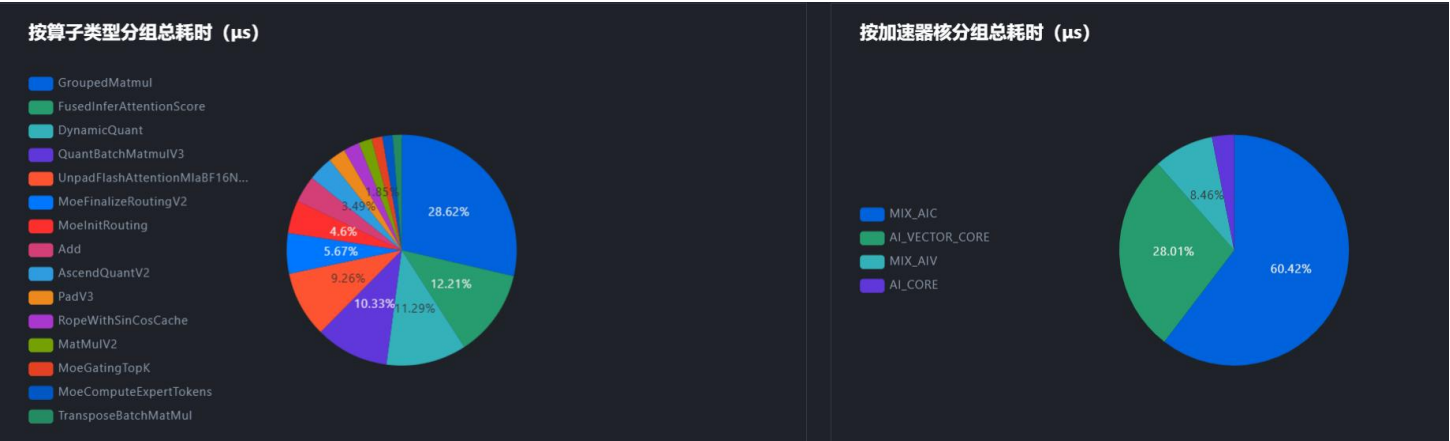
Operator视图 - 按类型统计 - 观察Top耗时算子、观察转换类算子占比

Operator页签

按类型统计

按shape统计

卡间比对



TIPS: 若想在Timeline中查看某个算子具体位置，可打开**Timeline页面 - 底部面板 - 系统视图 - 算子详情**，选择相应**卡序号**可按**名称、类型、加速器核、shape过滤**，按耗时**排序**，可跳转至时间线具体位置（比全局搜索更快）

选中详情 选中列表 系统视图 发现

统计系统视图

卡序号: 0

综合指标 ②
Python API 汇总
CANN API 汇总
Ascend HardWare Task 汇总
通信汇总
覆盖分析
算子详情

名称	类型	加速器核	开始时间	时长(us)	等待时间(us)	任务ID	Block数量	输入Shapes	输入数据...	输入格式	输出Shapes	输出数据...	输出格式	点击跳转Timeline
hcom_allReduc...	hcom_allReduce_	HCCL	1491ms155us8...	514792.46	1375.34	N/A	0	N/A	N/A	N/A	N/A	N/A	N/A	点击
hcom_allReduc...	hcom_allReduce_	HCCL	4359ms634us4...	317353.8	1615.28	N/A	0	N/A	N/A	N/A	N/A	N/A	N/A	点击
hcom_allReduc...	hcom_allReduce_	HCCL	5104ms257us1...	169735.54	17879.58	N/A	0	N/A	N/A	N/A	N/A	N/A	N/A	点击
hcom_allReduc...	hcom_allReduce_	HCCL	2488ms79us29...	169390.92	17490.82	N/A	0	N/A	N/A	N/A	N/A	N/A	N/A	点击
hcom_allReduc...	hcom_allReduce_	HCCL	2355ms531us3...	88294.44	0	N/A	0	N/A	N/A	N/A	N/A	N/A	N/A	点击
hcom_allReduc...	hcom_allReduce_	HCCL	4972ms357us9...	88178.46	0	N/A	0	N/A	N/A	N/A	N/A	N/A	N/A	点击
hcom_allReduc...	hcom_allReduce_	HCCL	2706ms840us8...	83492.16	40035.72	N/A	0	N/A	N/A	N/A	N/A	N/A	N/A	点击
hcom_allReduc...	hcom_allReduce_	HCCL	5323ms530us5...	82939.28	40216.64	N/A	0	N/A	N/A	N/A	N/A	N/A	N/A	点击
hcom_allReduc...	hcom_allReduce_	HCCL	4906ms241us5...	66112.64	0	N/A	0	N/A	N/A	N/A	N/A	N/A	N/A	点击
hcom_allReduc...	hcom_allReduce_	HCCL	2289ms646us1...	65881.2	24818.5	N/A	0	N/A	N/A	N/A	N/A	N/A	N/A	点击

共 39951 条 < 1 2 3 4 5 ... 3996 > 10 条/页 跳至 页

Operator页签

——观察Top耗时算子

Operator页签

按类型统计

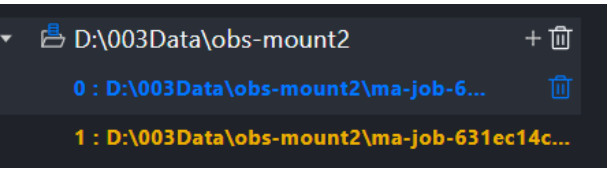
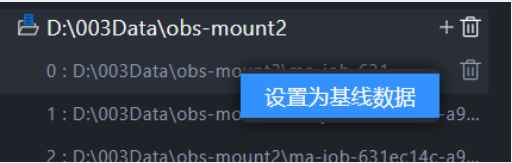
按shape统计

卡间对比

支持开启卡间对比

Step1：两卡Profiling移至同一父目录，Insight打开该父目录

Step2：右键子目录-选择基线数据



Case：计算快慢卡问题 - Operator视图 - 卡间比对

算子详情

类型	来源	加速器核	数量	总耗时(μs)	平均耗时(μs)	最大耗时(μs)	最小耗时(μs)	详情
MatMulV3	差值	MIX_AIC	0	-74697.96	-32.42	16.02	-9.18	查看更多
Add	差值	AI_VECTOR_CORE	-46	-16164.3	-1.98	-104.48	0.06	查看更多
MatMulV2	差值	AI_CORE	0	-15003.2	-6.51	-27.82	-1.1	查看更多
SwiGluGrad	差值	AI_VECTOR_CORE	0	-4163.12	-10.84	13.04	-10.26	查看更多
TensorMove	差值	AI_VECTOR_CORE	0	-2273.16	-2.84	-3.26	-2.44	查看更多
ZerosLike	差值	AI_VECTOR_CORE	0	-1797.62	-2.3	-191.82	0.04	查看更多
MemSet	差值	AI_VECTOR_CORE	-16	-1659.9	-1.29	-75.42	-0.02	查看更多
SwiGlu	差值	AI_VECTOR_CORE	0	-1583.26	-4.12	-11.92	-2.7	查看更多
Cast	差值	AI_VECTOR_CORE	-16	-1128.14	-0.26	-52.96	0.16	查看更多
FlashAttentionScore	差值	MIX_AIC	0	-1114.2	-2.9	78.58	0	查看更多

共 15 条 < 1 2 > 10 条/页

观察到计算耗时差异来源主要为MatMulV3算子，进一步按相同shape对比

算子详情

名称	来源	Shape	加速器核	数量	总耗时(μs)	平均耗时(μs)	最大耗时(μs)	最小耗时(μs)	详情
acInnMatmul_MatMulV3Common_MatMulV3	差值	"4096,7168;7168,12288"	MIX_AIC	0	-22858.44	-59.53	16.02	-53.84	查看更多
acInnMatmul_MatMulV3Common_MatMulV3	差值	"4096,12288;7168,12288"	MIX_AIC	0	-20819.9	-54.22	-45.22	-81.44	查看更多
acInnMatmul_MatMulV3Common_MatMulV3	差值	"4096,7168;4096,12288"	MIX_AIC	0	-14215.54	-37.02	-33.66	-47.14	查看更多
acInnMatmul_MatMulV3Common_MatMulV3	差值	"4096,12288;12288,3584"	MIX_AIC	0	-9677.18	-25.2	-12	-16.02	查看更多
acInnMatmul_MatMulCommon_MatMulV2	差值	"4096,1792;4096,12288"	AI_CORE	0	-6148.14	-16.01	-11.86	-15.6	查看更多
acInnMatmul_MatMulV3Common_MatMulV3	差值	"4096,12288;4096,3584"	MIX_AIC	0	-5469.62	-14.24	77.24	-12.92	查看更多
acInnMatmul_MatMulCommon_MatMulV2	差值	"4096,12288;12288,1536"	AI_CORE	0	-3632.62	-9.46	6.22	-5	查看更多
acInnMatmul_MatMulCommon_MatMulV2	差值	"4096,1792;1792,12288"	AI_CORE	0	-2987.02	-7.78	-7.56	-11.44	查看更多
acInnMatmul_MatMulV3Common_MatMulV3	差值	"4096,3584;12288,3584"	MIX_AIC	0	-1657.28	-4.31	-17.96	-2.78	查看更多
acInnMatmul_MatMulCommon_MatMulV2	差值	"4096,12288;4096,1536"	AI_CORE	0	-1245.3	-3.24	18.6	-1.1	查看更多
acInnMatmul_MatMulCommon_MatMulV2	差值	"4096,12288;1792,12288"	AI_CORE	0	-1101.92	-2.87	-27.82	5.3	查看更多
acInnMatmul_MatMulCommon_MatMulV2	差值	"4096,1536;12288,1536"	AI_CORE	0	111.8	0.29	3.3	0.46	查看更多

经排查，最终确认是HBM颗粒差异+Matmul计算带宽抢占HBM程度不同导致。

老师敲黑板

Mindstudio Insight整体定位思路Top-Down，由整体（集群）至局部（单卡）：

1. 从集群维度，有Summary/Communication页签

- 首先进入Summary页签，通过多卡计算/通信/调度比对，初步定界问题；
- 随后进入Communication页签，按通信域拆解，进一步定位慢卡/慢链路问题；确认异常卡/链路后，可直接根据通信算子跳转至Timeline定位具体代码；

2. 从单卡维度，有Timeline/Memory/Operator页签

- Timeline页签：呈现单卡/多卡 一级流水（框架）二级流水（CANN）device侧，HCCL通信，overlap整体分析，用于问题详细定位分析。
- Memory页签：精准定位到内存消耗大的进程或算子，削峰填谷，提升NPU内存利用率。
- Operator页签：用于分析device侧算子执行，可按类型、按shape统计，支持两卡间比对

目 录

一、大模型训练性能调优概述

二、Pytorch大模型训练调优工具介绍

三、性能调优问题定位案例

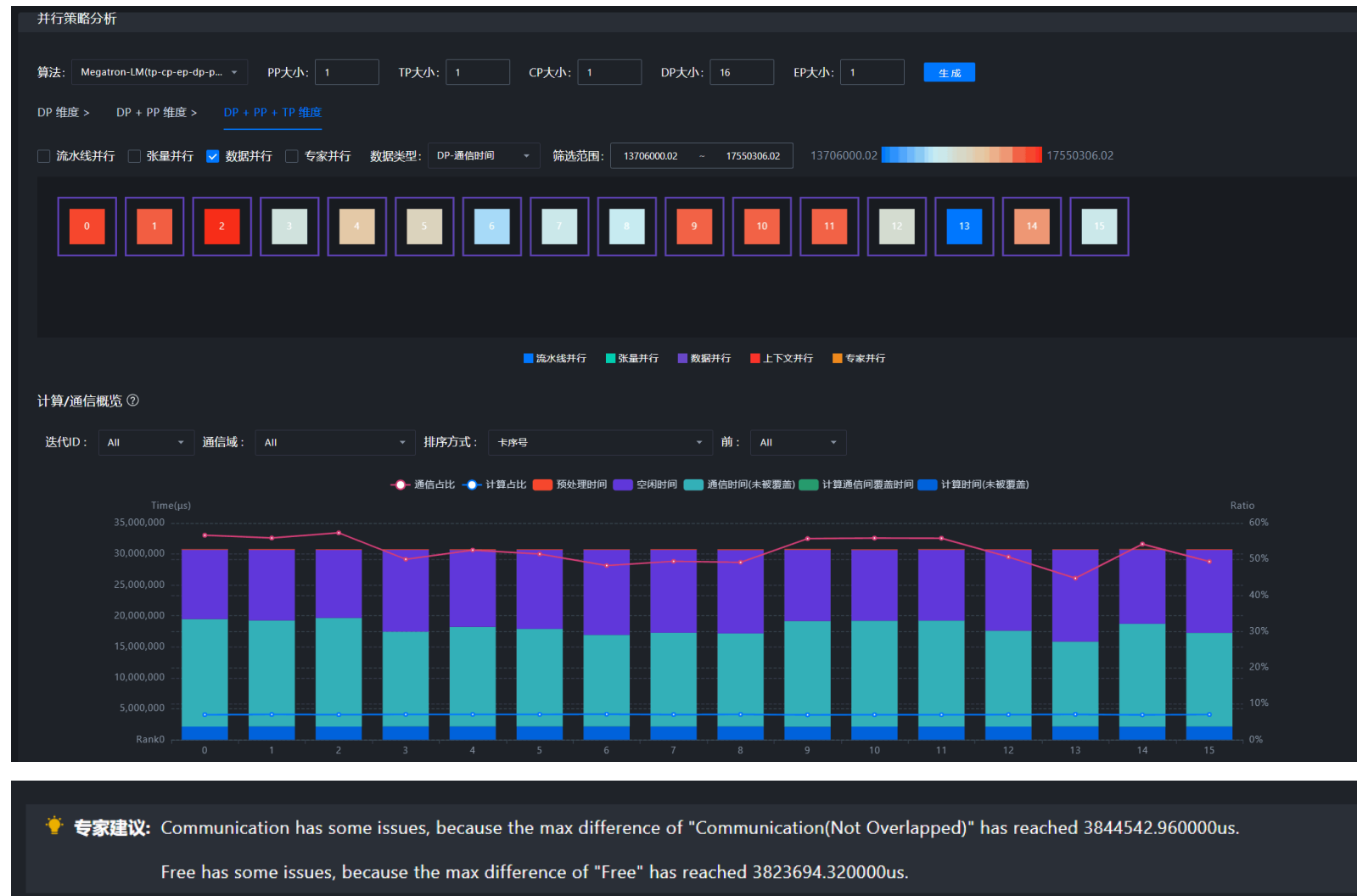
快慢卡定位-案例

Summary页签

右图是一个典型的

下发瓶颈 + 快慢卡 案例:

1. 计算占比低
2. 空闲/通信占比高
3. 各卡通信时间有波动。

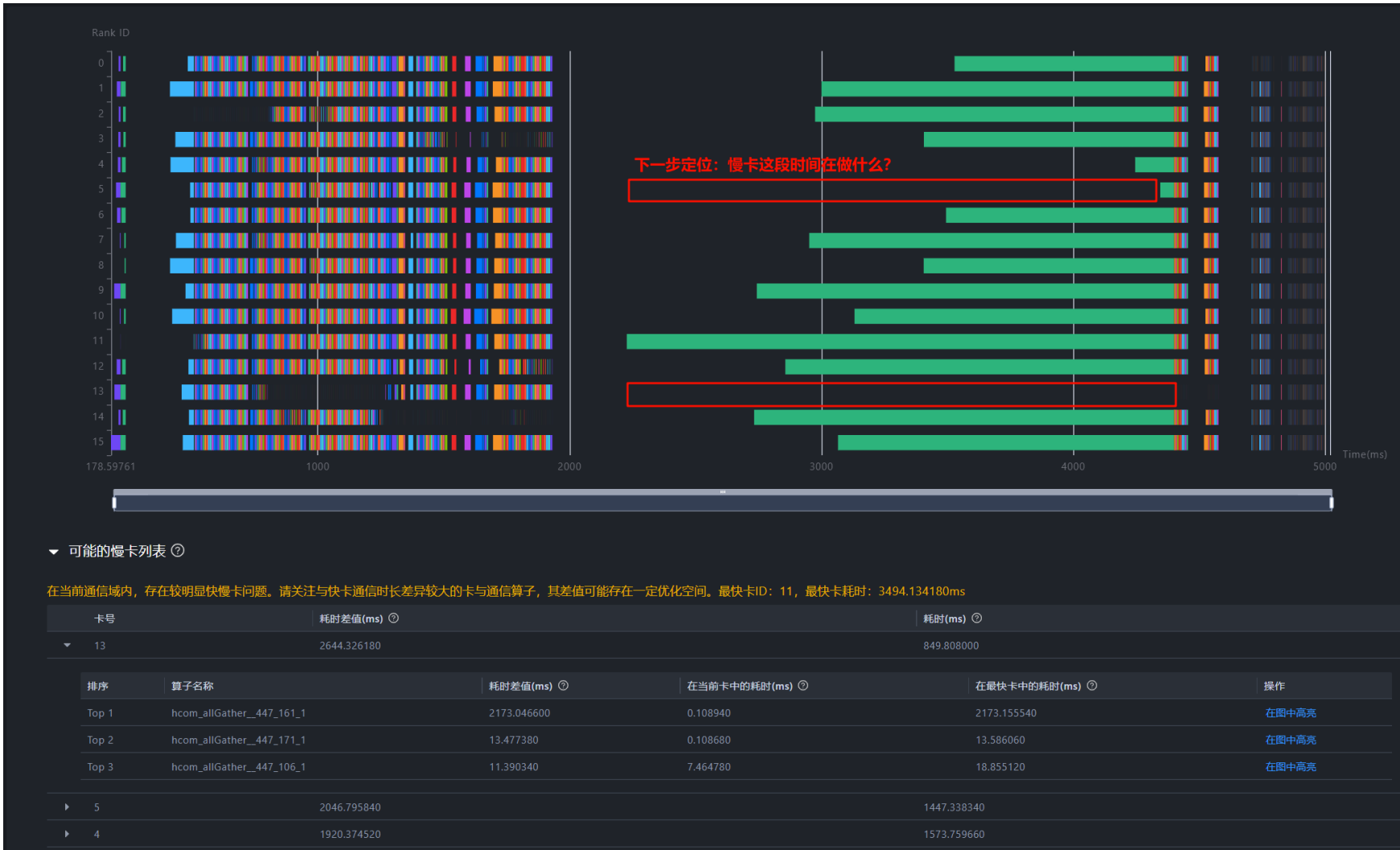


快慢卡定位-案例

Communication页签

右图是一个典型的
下发瓶颈+快慢卡案例：

- 1. 计算占比低
- 2. 空闲/通信占比高
- 3. 各卡通信时间有波动。



快慢卡问题：

“快慢卡”是相对的概念，快卡即集群中率先完成计算任务的卡。快慢卡不同步，一般表现为快卡存在耗时较长的通信算子，且等待时间占比较高。

慢卡、异常通信算子自动分析

Communication页签

Communication 页签

通信矩阵

通信耗时分析

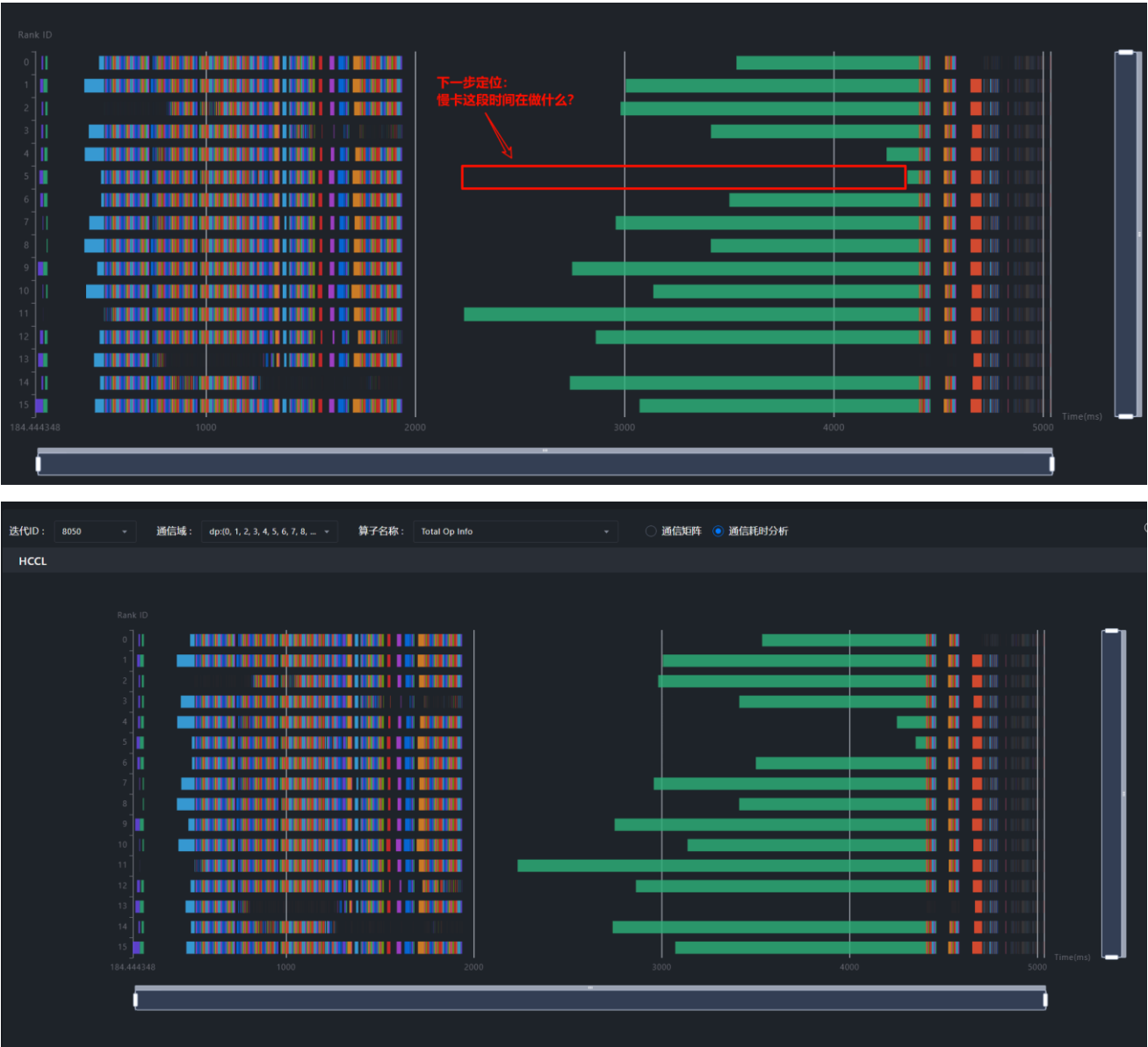
按通信域拆解

若传输时间占比低，等待/同步时间占比高，则为“**快慢卡问题**”

通信耗时分析将通信算子按时间横向平铺，可以清晰看到快慢卡对比情况。

下一步定位目标：分析慢卡在空白时间做什么？需至“**时间线**”模块，查看具体差异点。

👉 “通信” - “时间线” 两页签**根据通信算子互相跳转**（如右图所示），高效定位问题



Timeline页签

Timeline页签

置顶比对

连线调用关系

堆栈信息

按层过滤

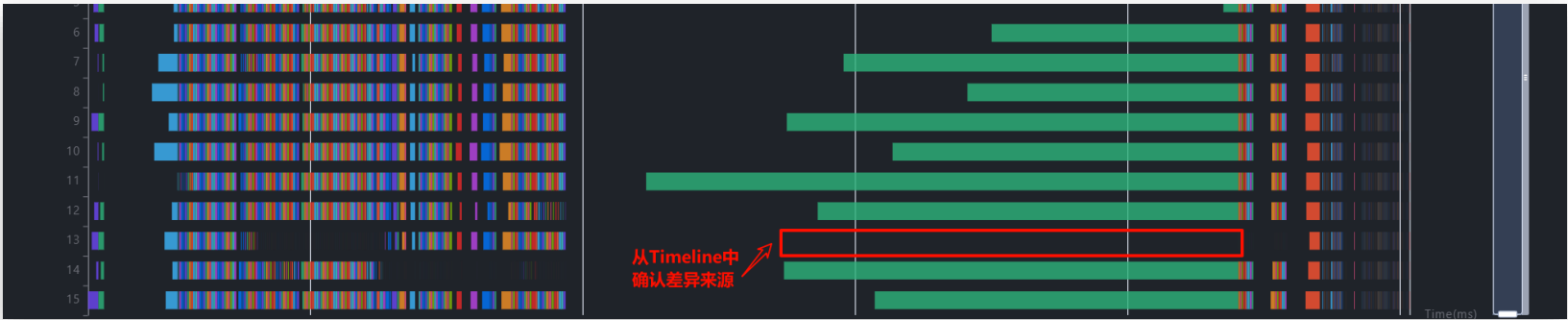
按区域框选统计

算子搜索

单卡统计信息

覆盖分析

专家建议



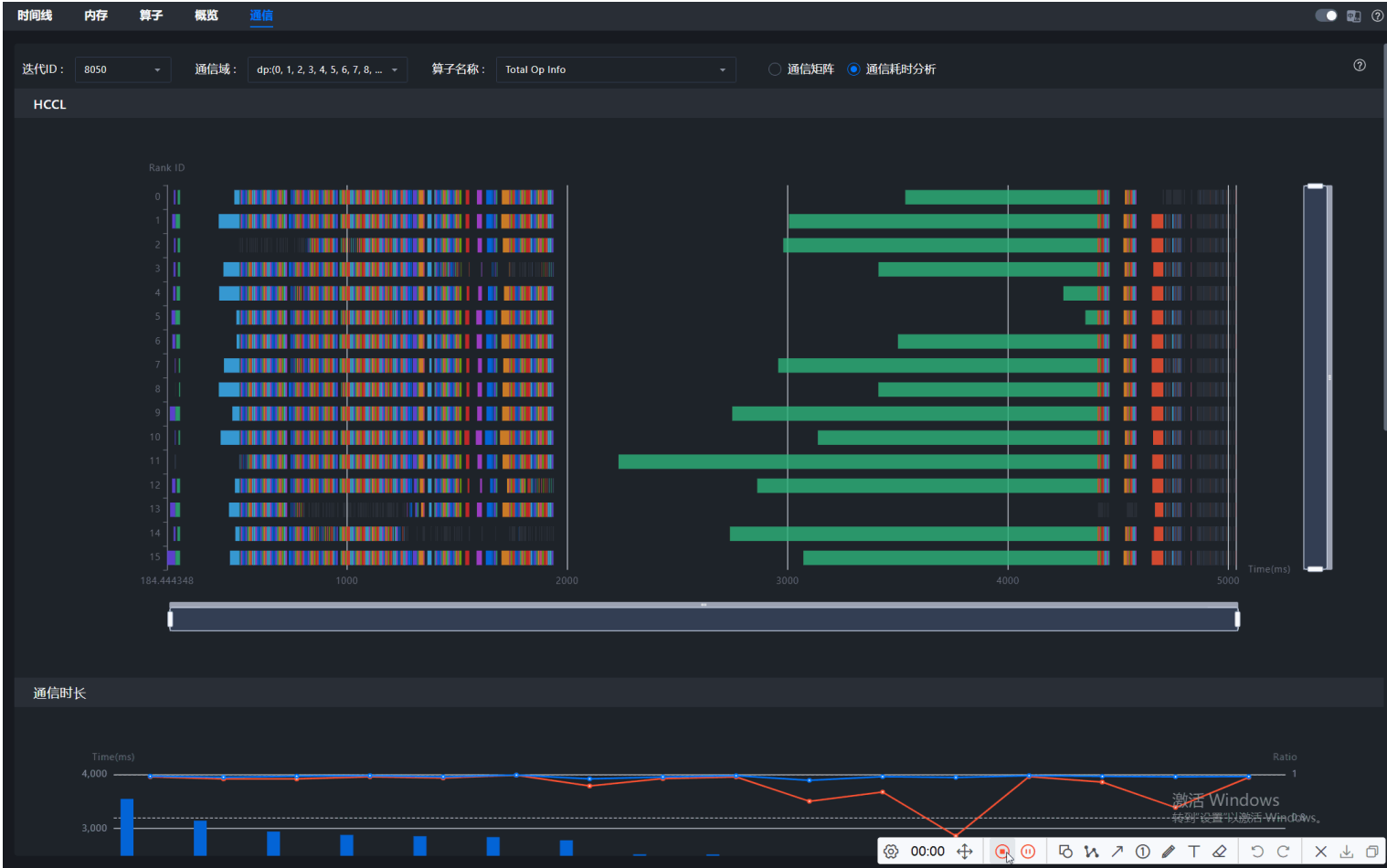
Step1. 由通信算子跳转至Timeline，用旗帜标记大致范围，方便后续定位。

Step2. 将研究区域限定在一个Step内，置顶对比两卡的覆盖分析泳道，确认差异来源。

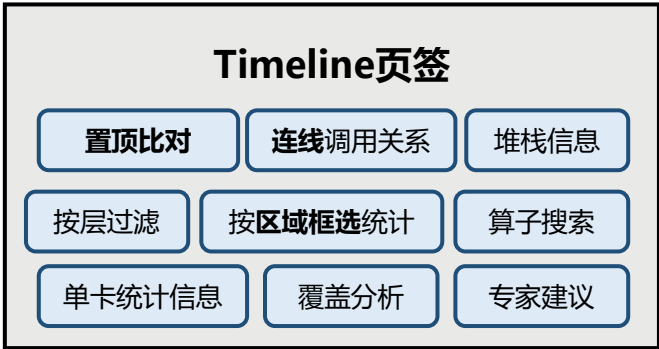
Step3. 框选统计可以看到，慢卡泳道计算任务次数明显多于快卡。

Step4. 根据async_npu下发连线，确认多出的算子由哪个Python API下发。

Step5. 和模型开发人员确认，该负载不均能否规避。



Timeline页签



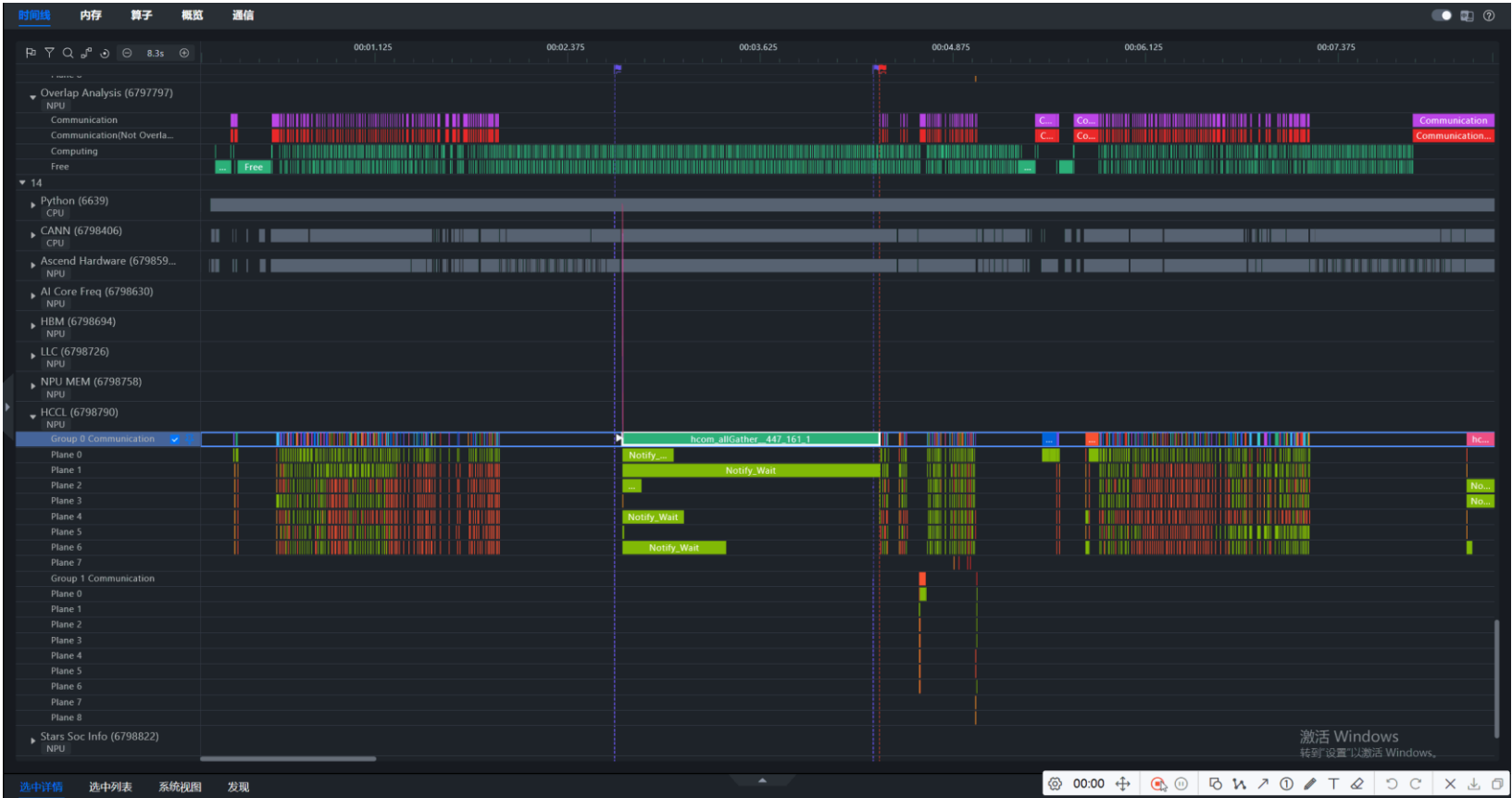
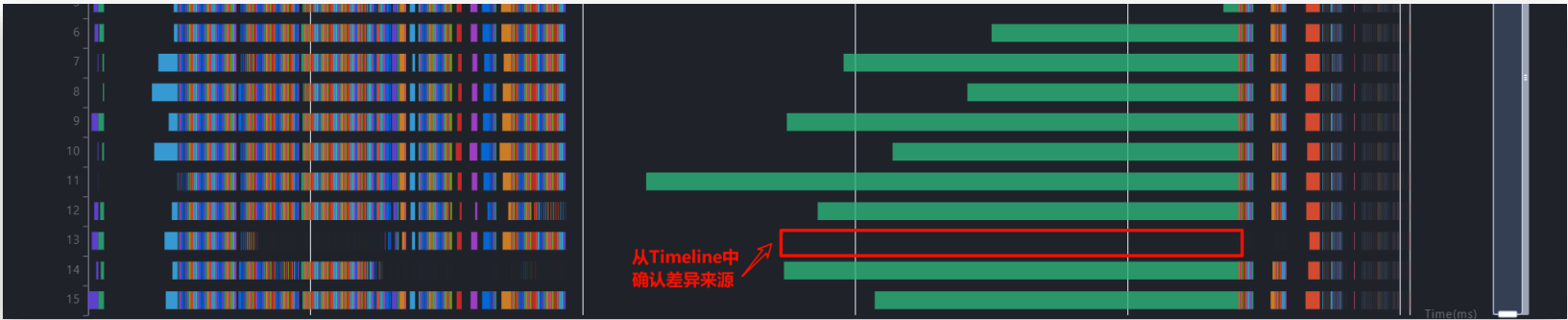
Step1. 由通信算子跳转至Timeline，用旗帜标记大致范围，方便后续定位。

Step2. 将研究区域限定在一个Step内，置顶对比两卡的覆盖分析泳道，确认差异来源。

Step3. 框选统计可以看到，慢卡泳道计算任务次数明显多于快卡。

Step4. 根据async_npu下发连线，确认多出的算子由哪个Python API下发。

Step5. 和模型开发人员确认，该负载不均能否规避。



Timeline页签

Timeline页签

置顶比对

连线调用关系

堆栈信息

按层过滤

按区域框选统计

算子搜索

单卡统计信息

覆盖分析

专家建议

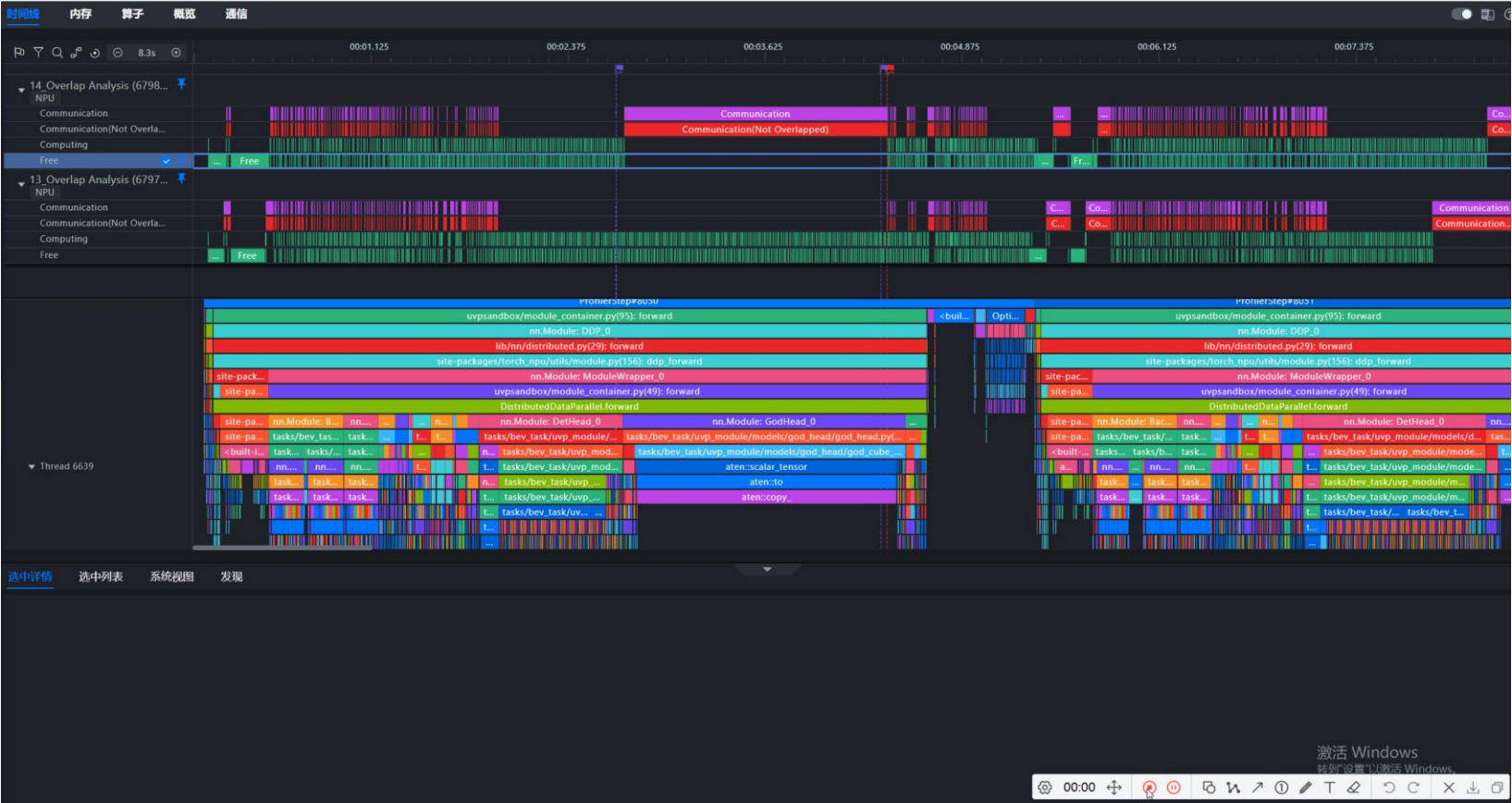
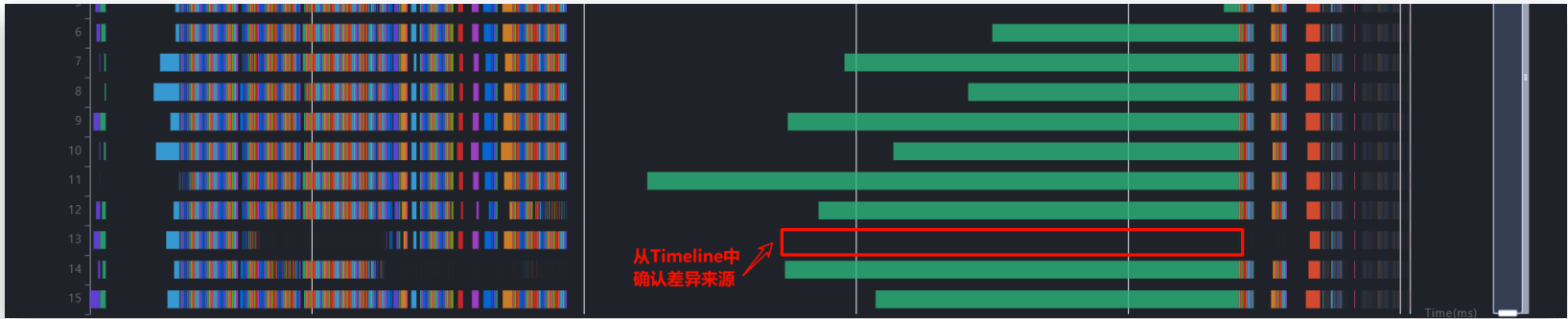
Step1. 由通信算子跳转至Timeline，用旗帜标记大致范围，方便后续定位。

Step2. 将研究区域限定在一个Step内，置顶对比两卡的覆盖分析泳道，确认差异来源。

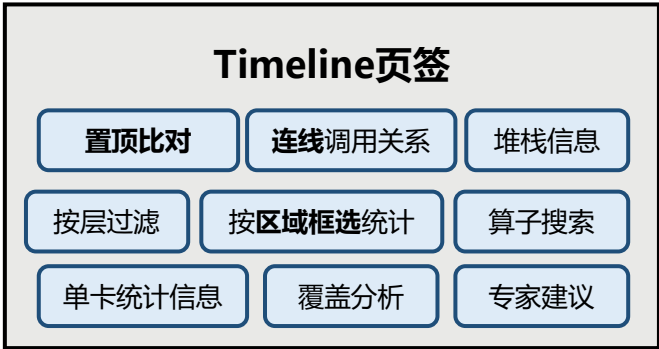
Step3. 框选统计可以看到，慢卡泳道计算任务次数明显多于快卡。

Step4. 根据async_npu下发连线，确认多出的算子由哪个Python API下发。

Step5. 和模型开发人员确认，该负载不均能否规避。



Timeline页签



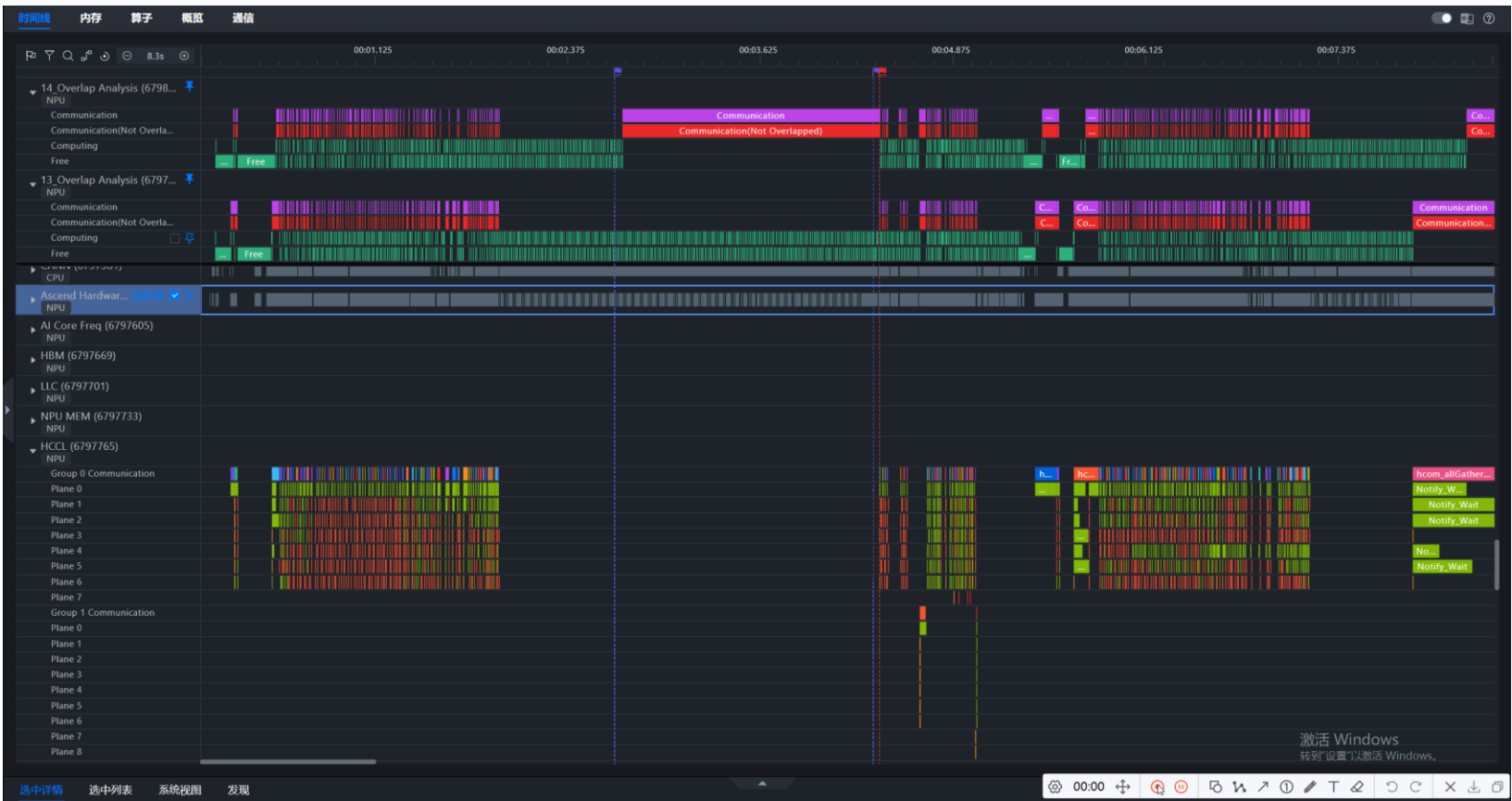
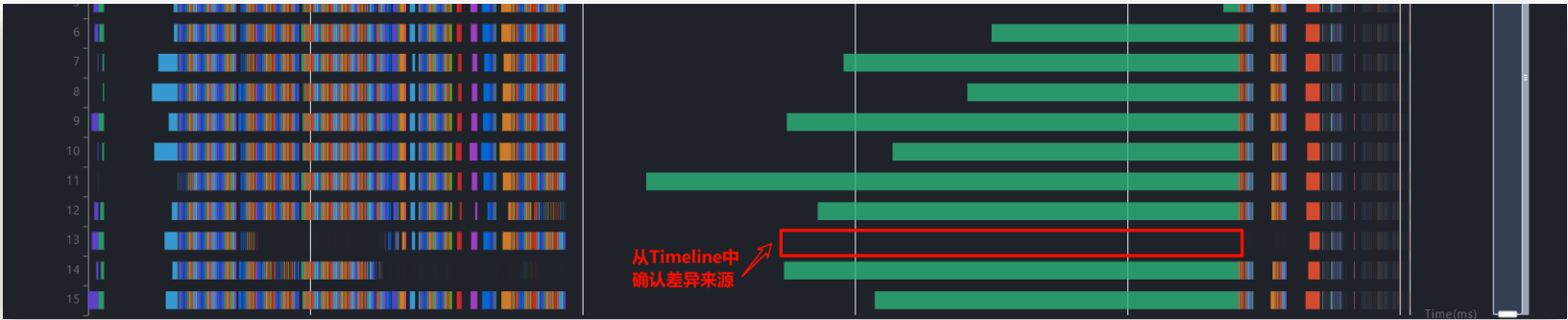
Step1. 由通信算子跳转至Timeline，用旗帜标记大致范围，方便后续定位。

Step2. 将研究区域限定在一个Step内，置顶对比两卡的覆盖分析泳道，确认差异来源。

Step3. 框选统计可以看到，慢卡泳道计算任务次数明显多于快卡。

Step4. 根据async_npu下发连线，确认多出的算子由哪个Python API下发。

Step5. 和模型开发人员确认，该负载不均能否规避。



Timeline页签

Timeline页签

置顶比对

连线调用关系

堆栈信息

按层过滤

按区域框选统计

算子搜索

单卡统计信息

覆盖分析

专家建议

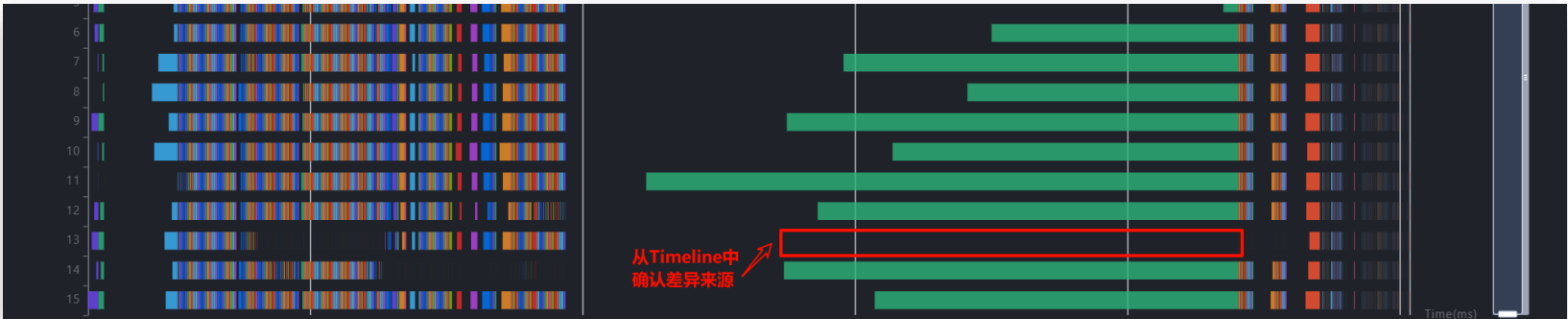
Step1. 由通信算子跳转至Timeline，用旗帜标记大致范围，方便后续定位。

Step2. 将研究区域限定在一个Step内，置顶对比两卡的覆盖分析泳道，确认差异来源。

Step3. 框选统计可以看到，慢卡泳道计算任务次数明显多于快卡。

Step4. 根据async_npu下发连线，确认多出的算子由哪个Python API下发。

Step5. 和模型开发人员确认，该负载不均能否规避。



对Ascend Hardware层框选统计得，同样的API tasks/bev_task/uvp_module/models/det_head/centerpoint/centerpoint_head.py(475): **get_targets_single**，快卡下发1218个计算算子

名称	持续时间	自用时间	平均持续时间	发生次数
Totals	4.232720 ms	4.232720 ms	0.003475 ms	1218

慢卡下发3303个计算算子

选中详情

选中列表

系统视图

发现

<

结论：get_targets_single函数负载不均导致快慢卡

总结页



- 性能优化的总体原则：**为减少Host算子下发时间和减少Device算子执行时间**
- **根据具体性能问题场景选择合适工具**
 - > **采集解析**：基于需求获取性能分析基础数据；
 - > **性能分析**：按问题场景进行性能对比，集群分析及调优分析；
 - > **可视化**：详细性能分析工具



性能工具总体介绍-MindStudio8.2.RC1-昇腾社区



Thanks !

大江东去浪淘沙 时代的洪流冲天下